

# GitHub Capstone - PROJECT STEP 1

## Detailed Click-by-Click Edition

From creating the shared repository to staging, committing, pushing and uploading Product V1 evidence - with explicit instructions for what to click, where to click, what to type and what you should see next.

### How this edition is different

Every screenshot-driven action includes: (1) the exact UI path, (2) numbered click instructions, (3) exact text/file names when required, (4) an expected-result checkpoint, and (5) a recovery note for common mistakes. This document covers Project Step 1 only and ends at its final sign-off.

### Scope of STEP 1

- 1 Shared GitHub repository and collaborators
- 2 Folder structure and project contract
- 3 Data Dictionary and 20-row starter data
- 4 Baseline pseudocode and local validator
- 5 Git staging, commit, authentication and push/sync
- 6 Product V1 sketch uploaded as repository evidence

# How to Read Every Action Page

Do not treat the screenshots as decoration. Use the click-path banner and the expected-result box as your navigation checkpoints.

## 1. WHERE TO CLICK

The blue banner tells you the exact screen region and menu path. Example: Repository page -> above file list -> Add file -> Create new file.

## 2. DO THIS - CLICK BY CLICK

Follow the numbered instructions in order. Do not skip a checkpoint because later screens assume the previous action succeeded.

## 3. TYPE / SELECT EXACTLY

Dark code boxes contain file paths, commit messages, commands or values that should be entered exactly as shown unless your project has an approved project-specific value.

## 4. EXPECTED RESULT

The green box tells you what must be visible after the action. If your screen does not match, stop before proceeding.

## 5. IF YOU DO NOT SEE THIS

Use the yellow recovery guidance. Most errors in this workflow are navigation, wrong-account, wrong-folder or uncommitted-change problems - not coding problems.

# Repository Creation & Team Setup

Create exactly one shared repository for the team, initialize it correctly, and invite every collaborator before anyone starts editing project files.

**By the end of this phase, students should be able to:**

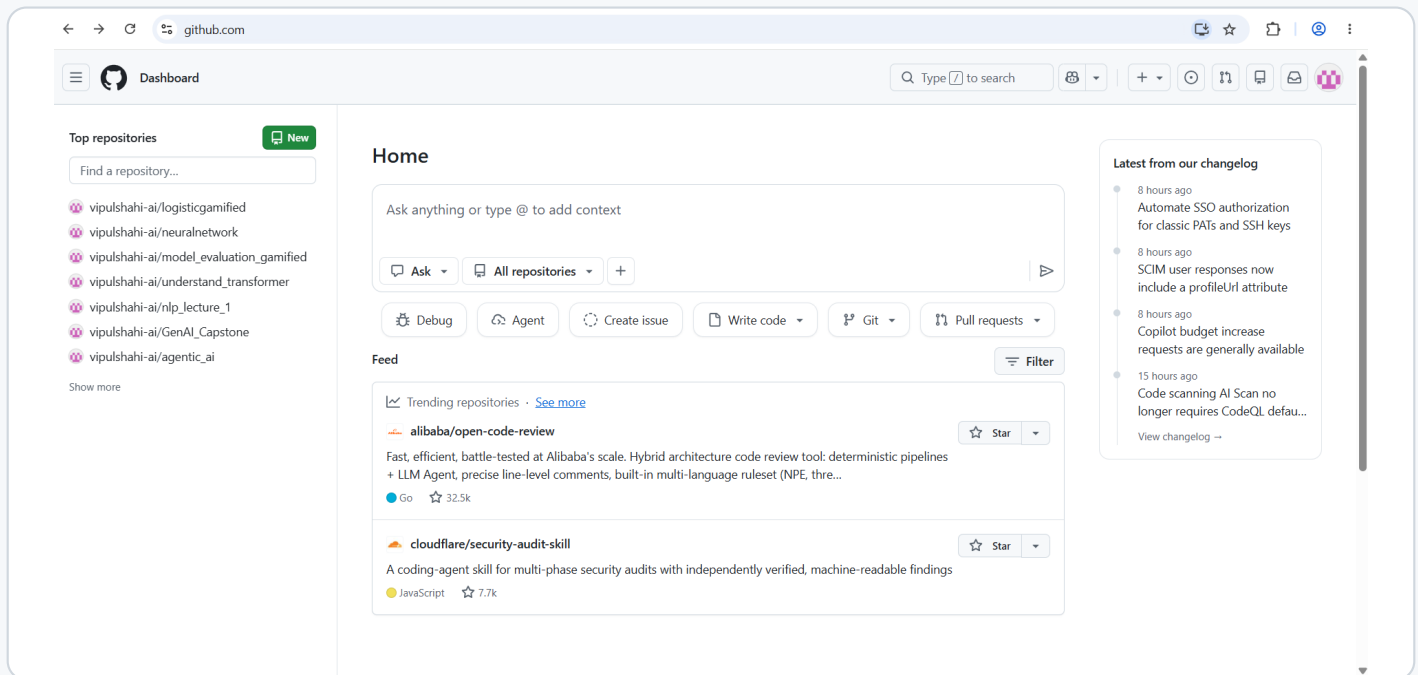
- 1 Create a public repository with README initialization.
- 2 Verify that the repository opens correctly and is visibly Public.
- 3 Open repository Settings and add all team members as collaborators.
- 4 Confirm every invitation is accepted before local Git work begins.

*Follow the screenshots and the click path exactly. Stop and verify each checkpoint before moving forward.*

# Start a New Repository

Begin from the signed-in GitHub dashboard.

**WHERE TO CLICK** GitHub dashboard -> left repository area -> green New button



Screenshot 102000 - GitHub dashboard

## DO THIS - CLICK BY CLICK

- 1 Open github.com and sign in to the GitHub account that will own the team repository.
- 2 On the dashboard, look at the left side where your repositories are listed.
- 3 Locate the green New button near the repository heading.
- 4 Click New once. Do not use New issue, New project, or a browser bookmark.
- 5 Wait for the Create a new repository page to load before entering any details.

### EXPECTED RESULT

You should be on a page titled Create a new repository with fields for owner, repository name, visibility and initialization options.

### IF YOU DO NOT SEE THIS

If New is not visible, use the + menu in the GitHub top bar and choose New repository. Make sure you are signed in to the intended GitHub account.

# Configure the Repository

Use a clear project name and initialize README from the first commit.

**WHERE TO CLICK** Create a new repository page -> Repository name / Visibility / Add a README file

Screenshot 102055 - repository configuration

## DO THIS - CLICK BY CLICK

- 1 Check the Owner field at the top. Leave it on the team member/account that should own the repository.
- 2 Click the Repository name field and enter the approved project repository name.
- 3 Under Visibility, select Public.
- 4 Find the initialization options and turn Add a README file ON.
- 5 Leave .gitignore and license unchanged unless your faculty/project instructions specifically require them.
- 6 Re-read the repository name for spelling and project-number errors before continuing.

### TYPE / SELECT EXACTLY

SC11-Water-Quality-Potability-Example

### EXPECTED RESULT

The form shows the correct repository name, Public selected, and Add a README file enabled.

### IF YOU DO NOT SEE THIS

If GitHub reports that the name already exists under the same owner, choose the approved unique team/project name. Do not silently add random suffixes to the official repository name.

# Create the Repository

Submit the form only after all required settings are correct.

**WHERE TO CLICK** Create a new repository page -> scroll to bottom -> green Create repository button

github.com/new

Great repository names are short and memorable. How about [ubiquitous-parakeet](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility \*  
Choose who can see and commit to this repository

Public

Start with a template  
Templates pre-configure your repository with files.

No template

Add README  
READMEs can be used as longer descriptions. [About READMEs](#)

On

Add .gitignore  
.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license  
Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Screenshot 102116 - Create repository button

## DO THIS - CLICK BY CLICK

- 1 Scroll to the bottom of the repository-creation form.
- 2 Before clicking, confirm the repository name one final time.
- 3 Confirm Public is still selected.
- 4 Confirm Add a README file is still enabled.
- 5 Click the green Create repository button once.
- 6 Wait for GitHub to finish creating the repository; do not refresh during creation.

## EXPECTED RESULT

GitHub should open the newly created repository page automatically.

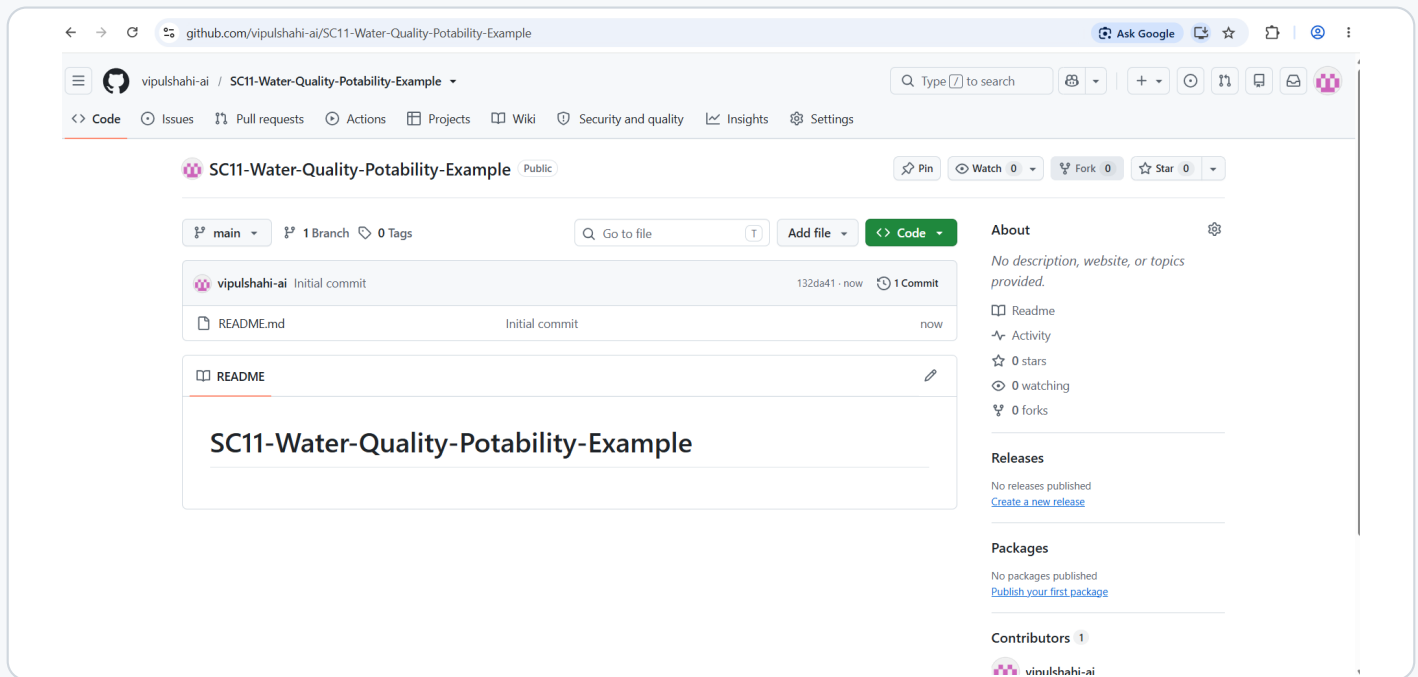
## IF YOU DO NOT SEE THIS

If the button is disabled, scroll upward and look for a validation message beside a required field, usually the repository name.

# Verify the Blank Workspace

Do not invite teammates until the new repository itself is correct.

**WHERE TO CLICK** New repository page -> file list / repository title / visibility badge



Screenshot 102141 - new repository

## DO THIS - CLICK BY CLICK

- 1 Look at the repository title near the top-left and verify the project name.
- 2 Confirm the Public visibility indicator is visible near the repository title.
- 3 In the file list, confirm README.md exists.
- 4 Open README.md only if needed; its presence is enough to prove initialization succeeded.
- 5 If the name or visibility is wrong, fix it now before inviting teammates and before adding project files.

### EXPECTED RESULT

The repository page shows the correct project name, Public visibility and README.md in the root file list.

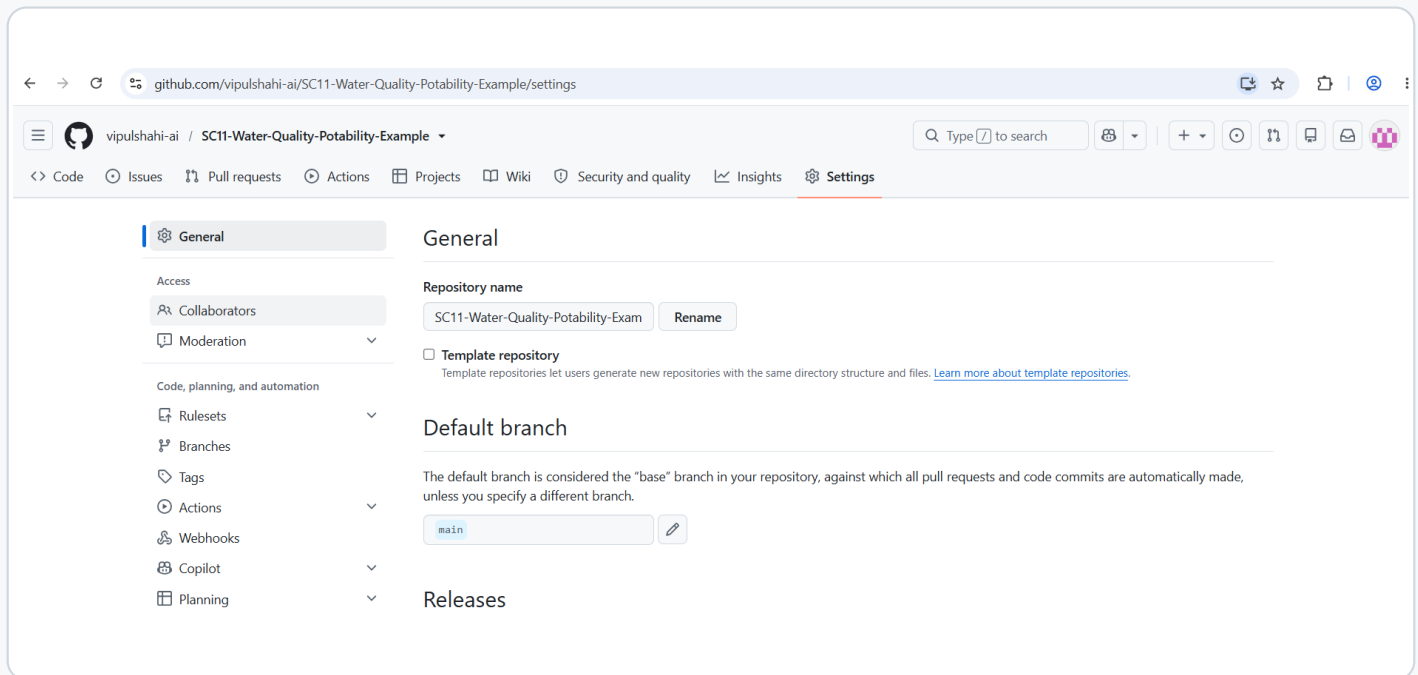
### IF YOU DO NOT SEE THIS

If README.md is missing, the repository was not initialized as intended. It is safer to correct the setup now than to build the remaining structure on an inconsistent starting point.

# Open Repository Settings

Repository collaboration permissions are managed from Settings, not from your personal account settings.

**WHERE TO CLICK** Repository page -> top navigation row -> Settings tab



Screenshot 121824 - repository Settings tab

## DO THIS - CLICK BY CLICK

- 1 Stay inside the team repository.
- 2 Look across the repository navigation row near Code, Issues, Pull requests, Actions and other tabs.
- 3 Click Settings at the far/right side of the repository navigation.
- 4 Wait for the repository Settings page to load.
- 5 Check the repository name in the page header to make sure you did not open global GitHub account settings by mistake.

### EXPECTED RESULT

You should see repository-specific Settings with a left sidebar containing collaboration/access options.

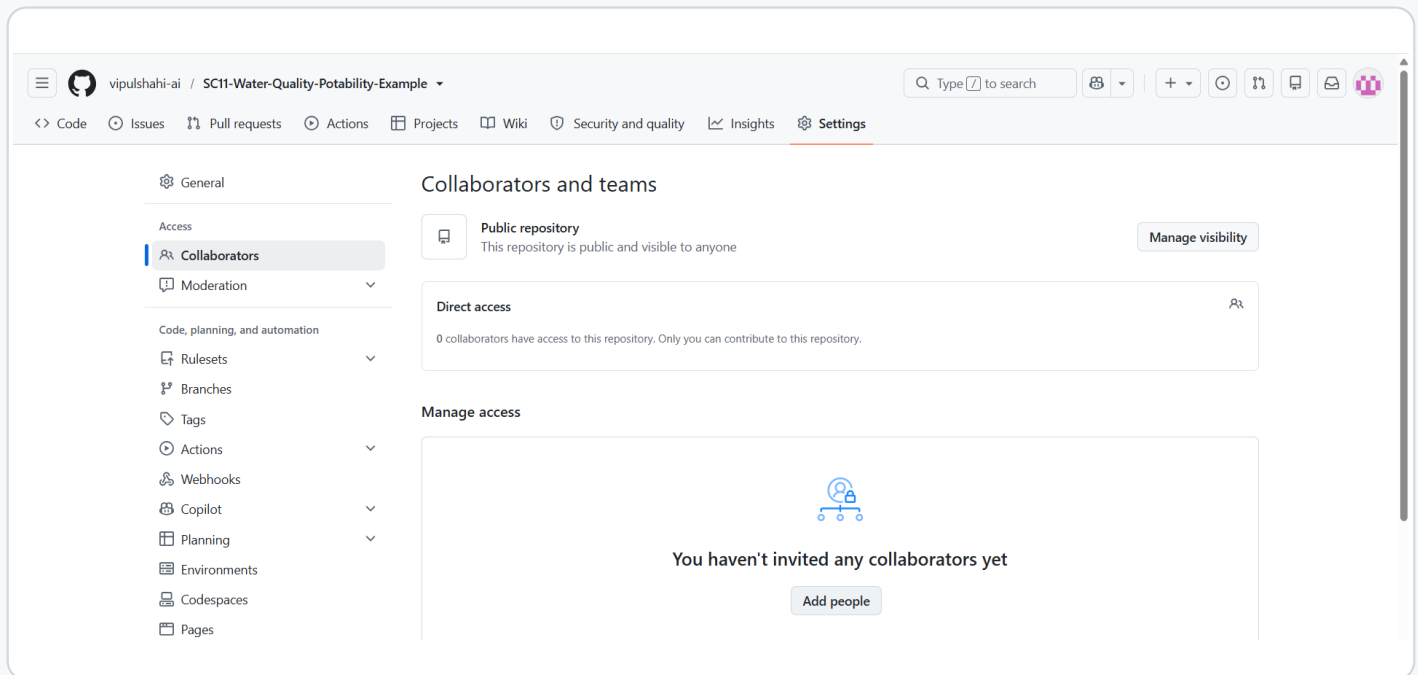
### IF YOU DO NOT SEE THIS

If Settings is not visible, you may not be the repository owner/admin. The student who created the repository should perform the collaborator setup.

# Open Collaborators and Add People

Give teammates write access to the shared repository.

**WHERE TO CLICK** Repository Settings -> left sidebar -> Collaborators -> Manage access -> Add people



Screenshot 121838 - Collaborators / Add people

## DO THIS - CLICK BY CLICK

- 1 In the left Settings sidebar, locate Collaborators (or Collaborators and teams depending on the GitHub UI).
- 2 Click Collaborators.
- 3 If GitHub asks you to confirm your password or authentication, complete it.
- 4 In Manage access, locate the green Add people button.
- 5 Click Add people to open the collaborator search dialog.

### EXPECTED RESULT

A search dialog should open where you can find a GitHub user by username or email.

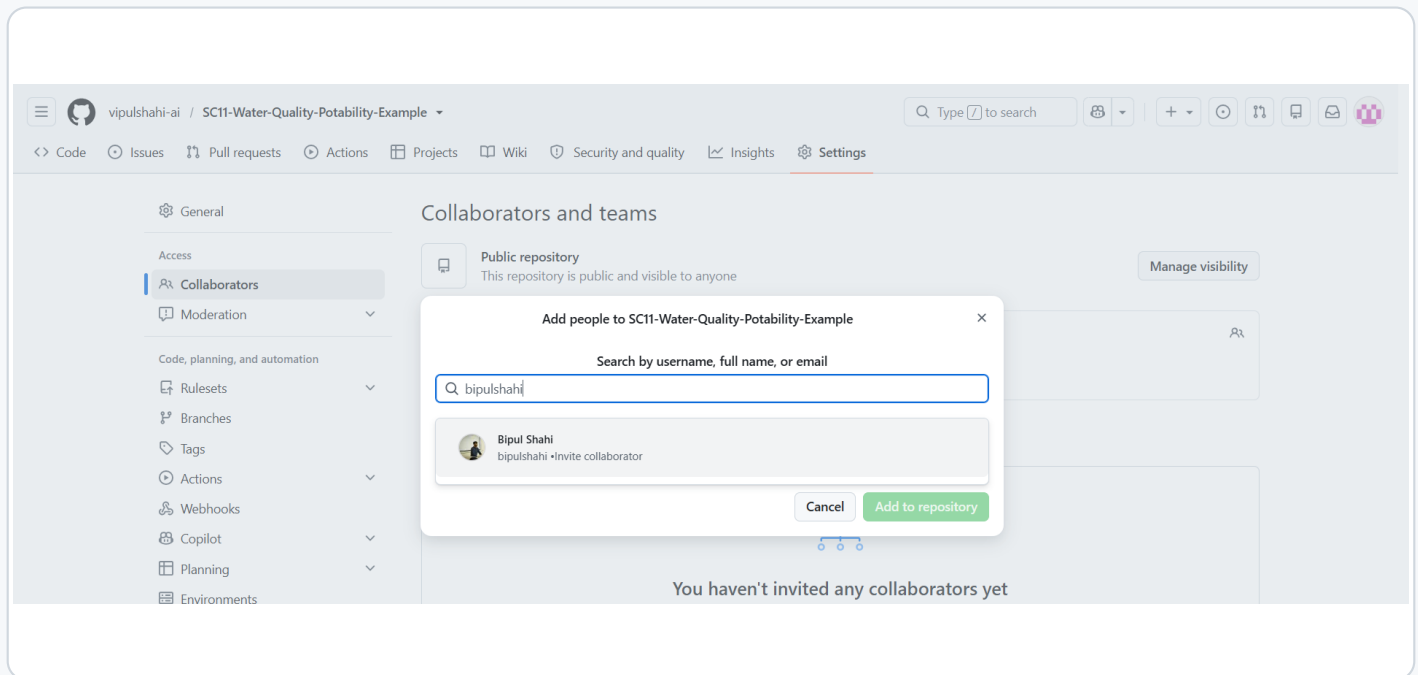
### IF YOU DO NOT SEE THIS

If Collaborators is missing, verify that you are in the repository Settings and that your account has admin permission for this repository.

# Find the Correct Teammate

Select the GitHub identity carefully; a similar display name can belong to another person.

**WHERE TO CLICK** Add people dialog -> search field -> result list



Screenshot 121856 - collaborator search

## DO THIS - CLICK BY CLICK

- 1 Click inside the collaborator search field.
- 2 Type the teammate's exact GitHub username whenever possible. An email linked to GitHub can also work.
- 3 Pause until GitHub returns matching accounts.
- 4 Compare the username/avatar with the teammate's confirmed GitHub account.
- 5 Click the correct result once.
- 6 Do not invite a guessed account just because the display name looks familiar.

### EXPECTED RESULT

The selected teammate should appear in the confirmation state of the Add people dialog.

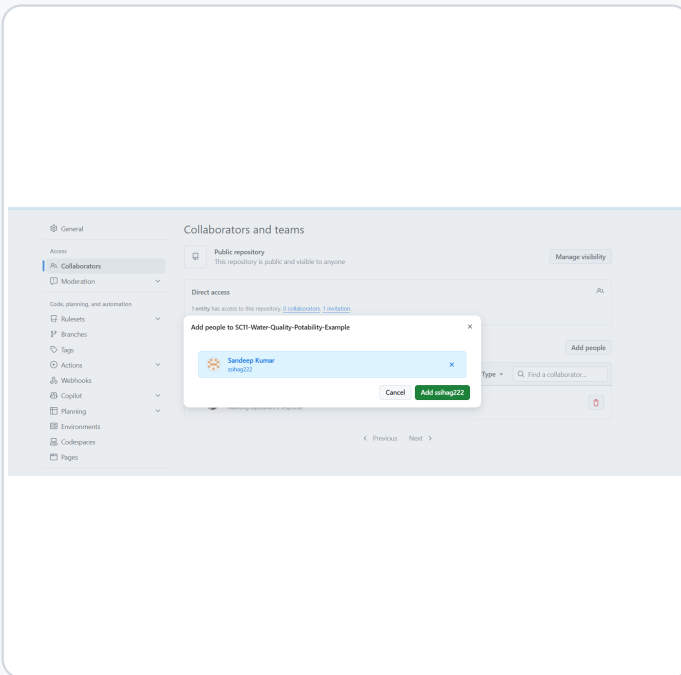
### IF YOU DO NOT SEE THIS

If no result appears, ask the teammate to send their exact GitHub username from their profile URL instead of repeatedly trying different emails.

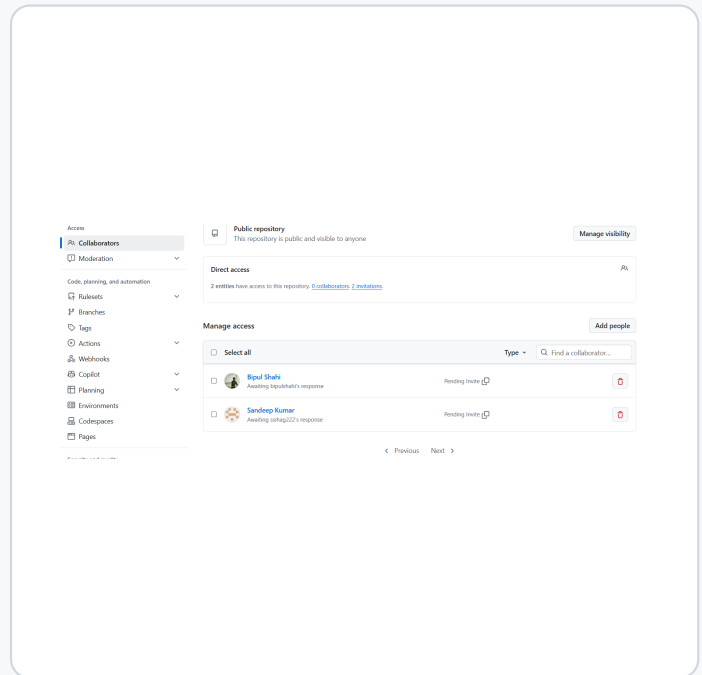
# Send the Invite and Confirm Acceptance

An invitation is not usable access until the teammate accepts it.

**WHERE TO CLICK** Add people confirmation -> green Add <username> button -> Manage access list



Screenshot 121952 - confirm Add user



Screenshot 122002 - Pending Invite

## DO THIS - CLICK BY CLICK

- 1 After selecting the correct account, click the green Add <username> button.
- 2 Return to Manage access and locate that teammate in the collaborator list.
- 3 If the status says Pending Invite, tell the teammate to check GitHub notifications and the email linked to GitHub.
- 4 The teammate must open the invitation and click Accept invitation.
- 5 Repeat the Add people process for every remaining teammate.
- 6 Before coding begins, re-open Manage access and confirm no required teammate is still pending.

## EXPECTED RESULT

All required team members are listed under repository access and have accepted their invitations. The team size should match the approved 3-4 student team.

## IF YOU DO NOT SEE THIS

A teammate with Pending Invite may be able to view a public repository but still cannot push. Do not confuse public viewing with collaborator write access.

## IMPORTANT

This collaborator acceptance matters later in Phase 5. A wrong or unaccepted account is a common reason for VS Code push-permission errors.

# Build the Folder Structure

Use the GitHub web interface to create the required folders without a terminal. The forward slash in a file path is what creates a folder in this workflow.

## By the end of this phase, students should be able to:

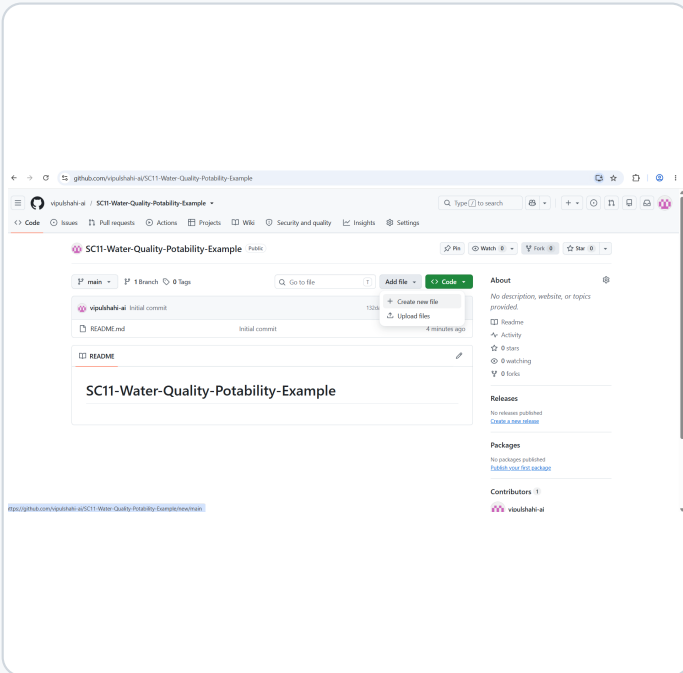
- 1 Create docs/STEP-1.md using GitHub's Create new file interface.
- 2 Create data/README.md from the repository root.
- 3 Commit each change directly to main for this Step-1 setup workflow.
- 4 Verify the root tree before moving into documentation.

*Follow the screenshots and the click path exactly. Stop and verify each checkpoint before moving forward.*

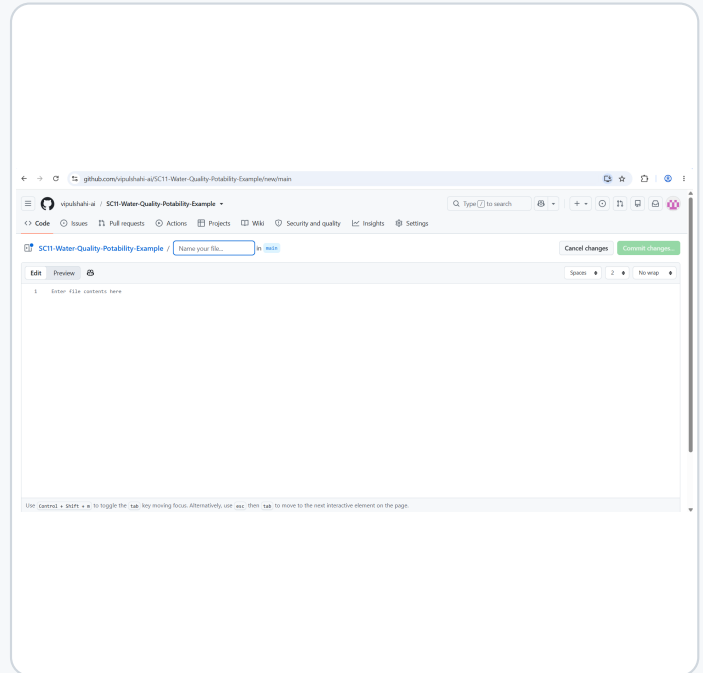
# Open GitHub's File Creator

Start from the repository root, not from inside README.md or another folder.

**WHERE TO CLICK** Repository root -> above file list -> Add file -> Create new file



Screenshot 102606 - Add file menu



Screenshot 102641 - new file editor

## DO THIS - CLICK BY CLICK

- 1 Open the repository Code page.
- 2 Make sure the breadcrumb shows only the repository root, not data/ or docs/.
- 3 Above the file list, locate Add file near the green Code button.
- 4 Click Add file to open its dropdown.
- 5 Click Create new file.
- 6 Confirm the page now contains a Name your file... field above a large editor area.

## EXPECTED RESULT

GitHub opens the browser file editor with a Name your file... field.

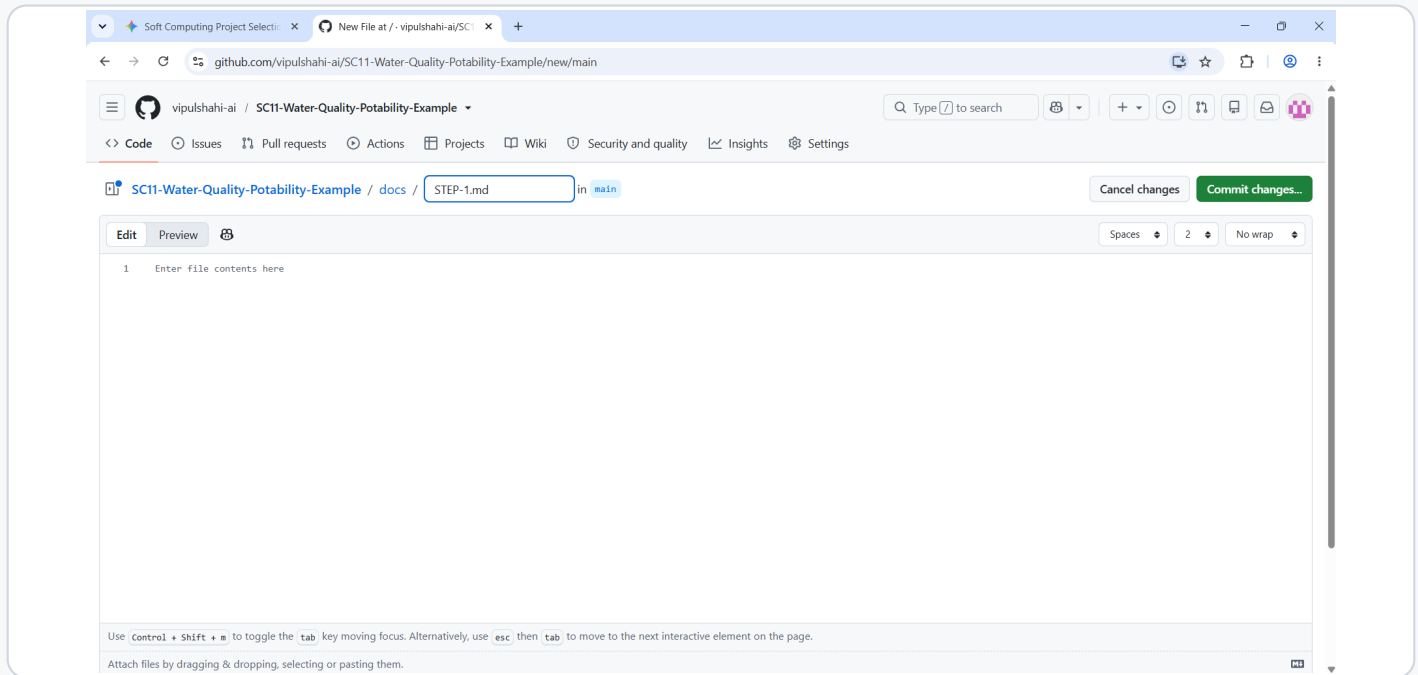
## IF YOU DO NOT SEE THIS

If you only see upload controls, you clicked Upload files instead of Create new file. Use the browser Back button and choose Create new file.

# Create docs/STEP-1.md

Typing the slash creates the docs folder and the file in one operation.

**WHERE TO CLICK** Create new file page -> Name your file... field



Screenshot 102819 - docs/STEP-1.md path

## DO THIS - CLICK BY CLICK

- 1 Click inside Name your file....
- 2 Type docs, then type a forward slash /.
- 3 GitHub should visually convert docs into a folder breadcrumb segment.
- 4 Continue typing STEP-1.md after the slash.
- 5 Check the final path carefully. It must be docs/STEP-1.md.
- 6 Leave the main file body blank at this stage unless your workflow already has the approved contract content ready.

### TYPE / SELECT EXACTLY

**docs/STEP-1.md**

### EXPECTED RESULT

The filename area shows docs / STEP-1.md, indicating GitHub will create the docs folder and file.

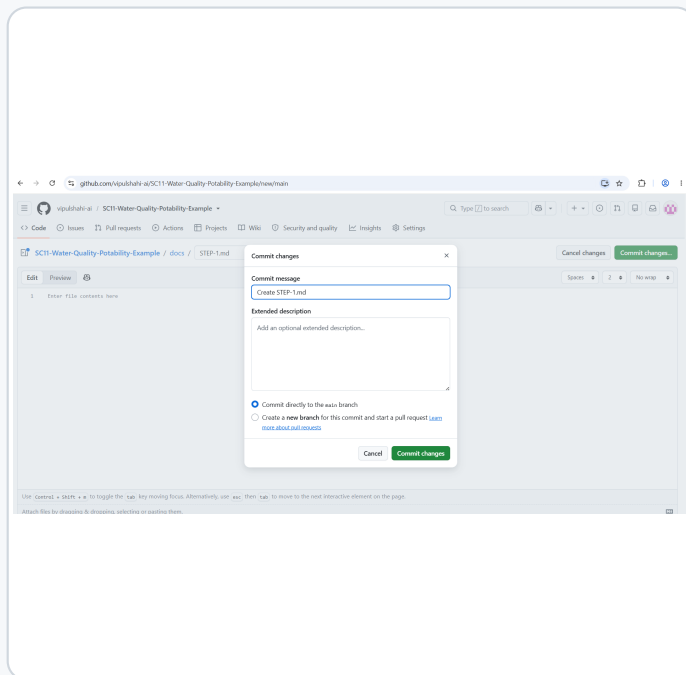
### IF YOU DO NOT SEE THIS

If the path reads STEP-1.md at root, you missed the docs/ prefix. Fix the filename before committing.

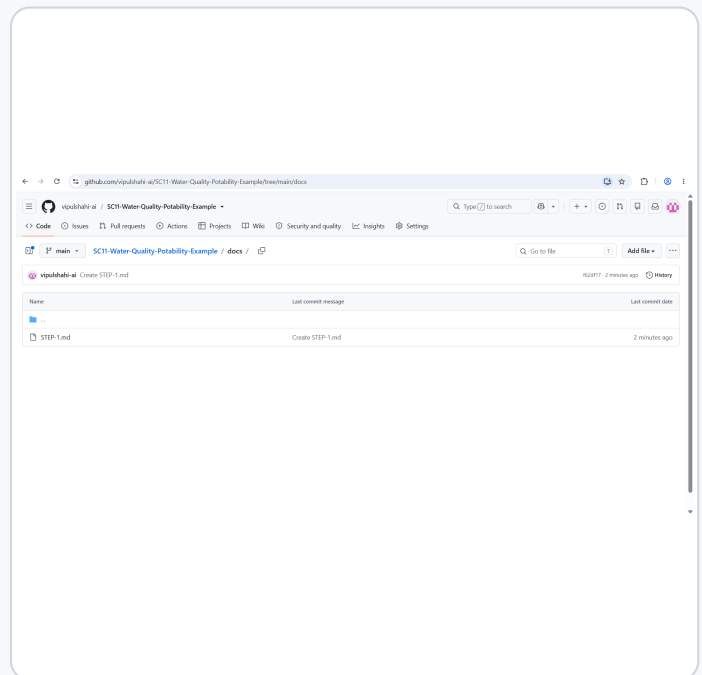
# Commit and Verify docs

A folder does not exist in Git until at least one file inside it is committed.

**WHERE TO CLICK** New file editor -> top-right Commit changes... -> commit dialog -> Commit changes



Screenshot 102834 - commit STEP-1.md



Screenshot 103057 - docs folder

## DO THIS - CLICK BY CLICK

- 1 Click Commit changes... at the top-right of the editor.
- 2 In the dialog, review the commit message. A simple create message is acceptable for this placeholder.
- 3 Keep Commit directly to the main branch selected for this guided Step-1 setup.
- 4 Click the green Commit changes button in the dialog.
- 5 After GitHub returns to the repository, click/open docs if needed.
- 6 Verify STEP-1.md appears inside docs/.

### EXPECTED RESULT

The docs folder exists and contains STEP-1.md on the main branch.

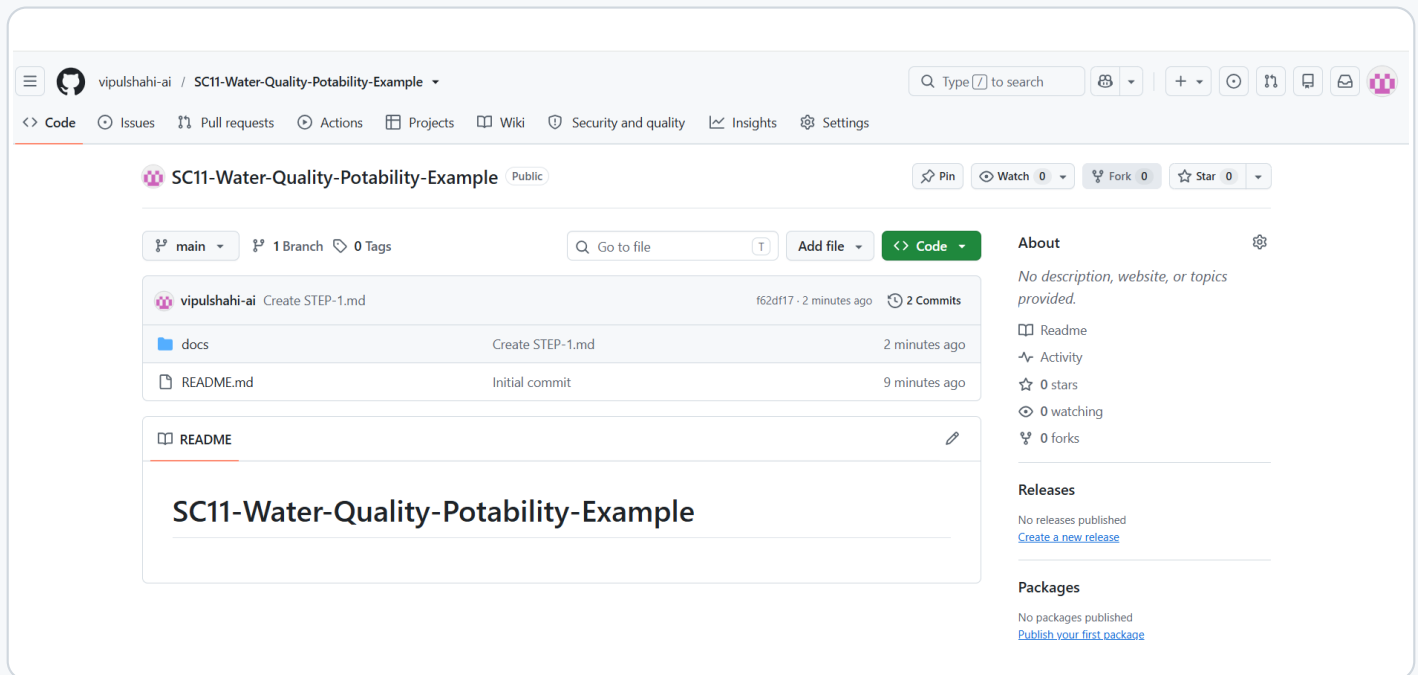
### IF YOU DO NOT SEE THIS

If docs appears empty or is missing, check the commit history and verify you actually completed the second Commit changes confirmation button.

# Return to Repository Root

Do not accidentally create data inside docs.

**WHERE TO CLICK** Current folder view -> breadcrumb near top-left -> repository name



Screenshot 103119 - repository breadcrumb/root

## DO THIS - CLICK BY CLICK

- 1 Look at the breadcrumb directly above the file list.
- 2 Click the repository name/first breadcrumb segment to return to root.
- 3 Confirm the file list shows README.md and docs/ at the same level.
- 4 Only after you are back at root, click Add file -> Create new file again.

### EXPECTED RESULT

You are back at the repository root and ready to create data/ as a sibling of docs/.

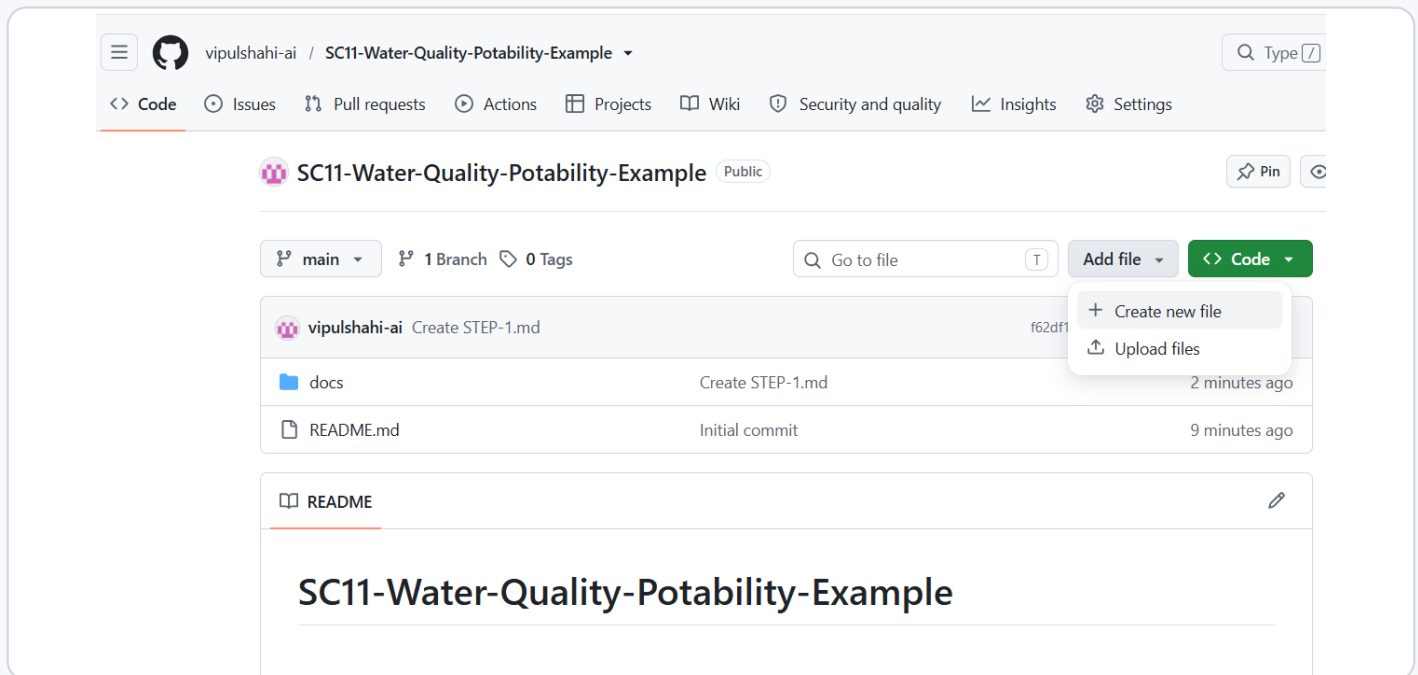
### IF YOU DO NOT SEE THIS

If the breadcrumb still includes docs, do not create the next file yet. Return to root first or you will create docs/data/... by mistake.

# Create data/README.md

The data folder gets a small README placeholder so GitHub can track the folder.

**WHERE TO CLICK** Repository root -> Add file -> Create new file -> Name your file...



Screenshot 103145 - data/README.md

## DO THIS - CLICK BY CLICK

- 1 Click Add file above the root file list.
- 2 Choose Create new file.
- 3 Click Name your file....
- 4 Type data/README.md exactly.
- 5 Confirm GitHub shows data as a folder segment and README.md as the file.
- 6 Leave the body blank or minimal for now; the formal Data Dictionary in this example will be stored in the root README.md.

### TYPE / SELECT EXACTLY

**data/README.md**

### EXPECTED RESULT

The filename path is data/README.md and data is being created at repository root level.

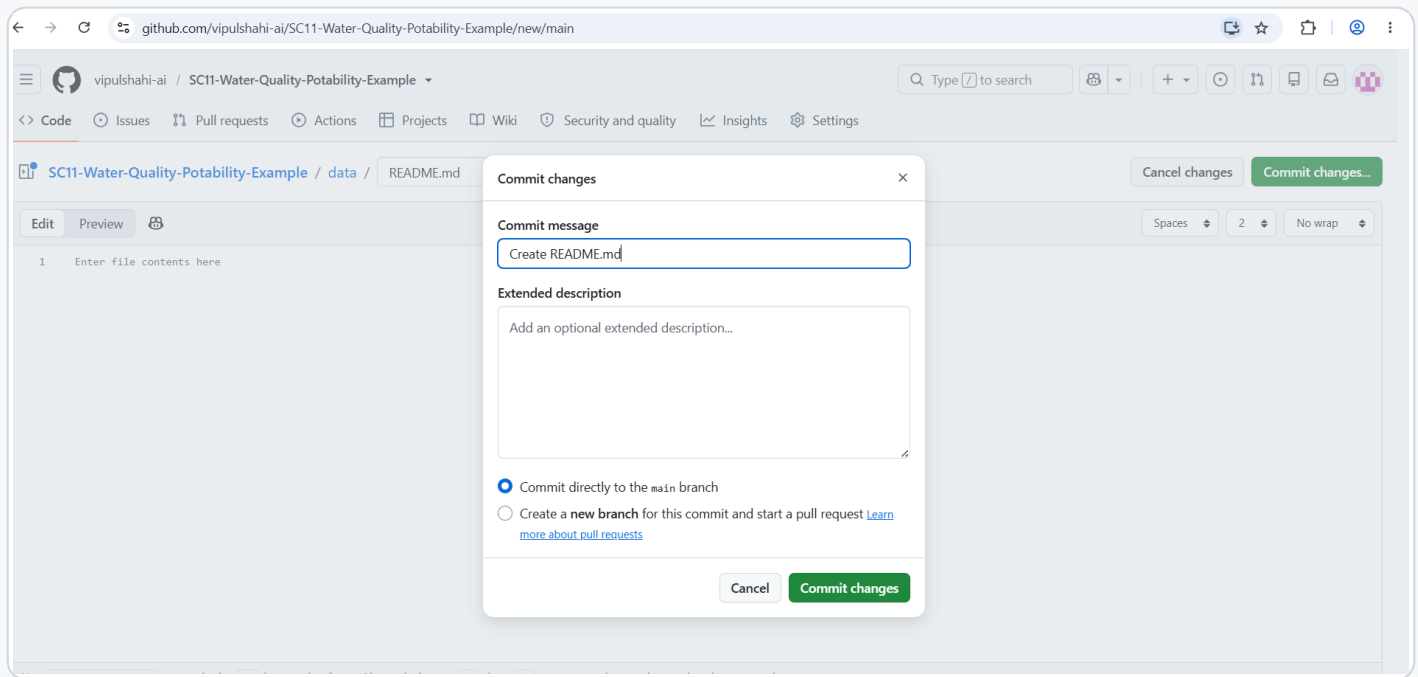
### IF YOU DO NOT SEE THIS

If the breadcrumb before the filename already contains docs, cancel and return to repository root before retrying.

# Commit data/README.md

Save the second folder placeholder to main.

**WHERE TO CLICK** New file editor -> Commit changes... -> commit dialog -> Commit changes



Screenshot 103232 - commit data README

## DO THIS - CLICK BY CLICK

- 1 Click Commit changes... at the top-right.
- 2 Review the commit message, for example Create README.md.
- 3 Keep Commit directly to the main branch selected.
- 4 Click the final green Commit changes button.
- 5 Wait for GitHub to return to the repository.

### EXPECTED RESULT

data/README.md is committed on main.

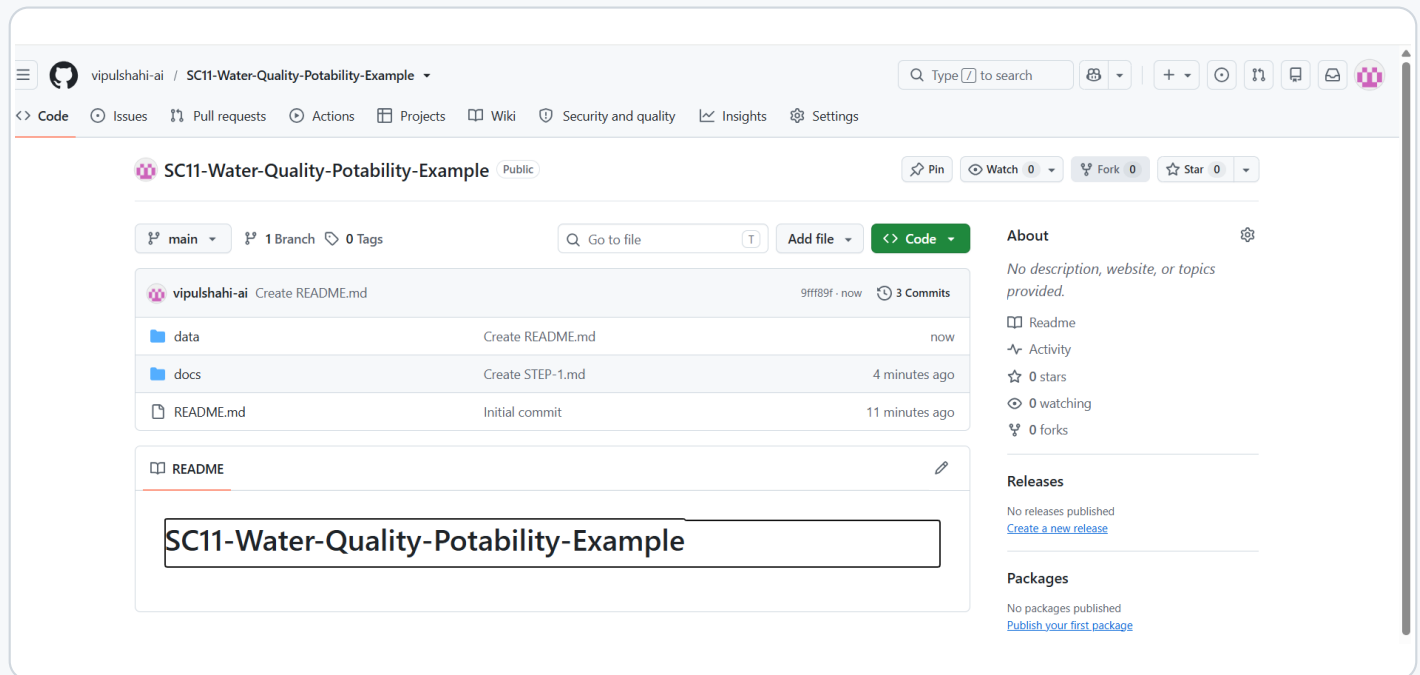
### IF YOU DO NOT SEE THIS

If GitHub asks you to create a pull request instead, check whether you changed the commit destination. For this guided initial setup, use direct-to-main as shown.

# Verify the Root Structure

Stop here if any folder is missing or nested incorrectly.

**WHERE TO CLICK** Repository Code page -> root file list



Screenshot 103246 - root tree

## DO THIS - CLICK BY CLICK

- 1 Click the repository name in the breadcrumb to return to root.
- 2 Look for data/ and docs/ in the same file list.
- 3 Confirm README.md is still present at root.
- 4 Open data/ and confirm README.md exists.
- 5 Open docs/ and confirm STEP-1.md exists.
- 6 Return to root before Phase 3.

## EXPECTED RESULT

Root tree: data/ , docs/ , README.md. Inside data/: README.md. Inside docs/: STEP-1.md.

## IF YOU DO NOT SEE THIS

If data or docs is nested inside the other, delete/recreate the misplaced path before continuing. Folder mistakes become harder to diagnose later.

# Project Documentation & Starter Data

Turn the empty structure into a shared project specification: Project Contract, Data Dictionary, starter CSV and implementation placeholders.

## By the end of this phase, students should be able to:

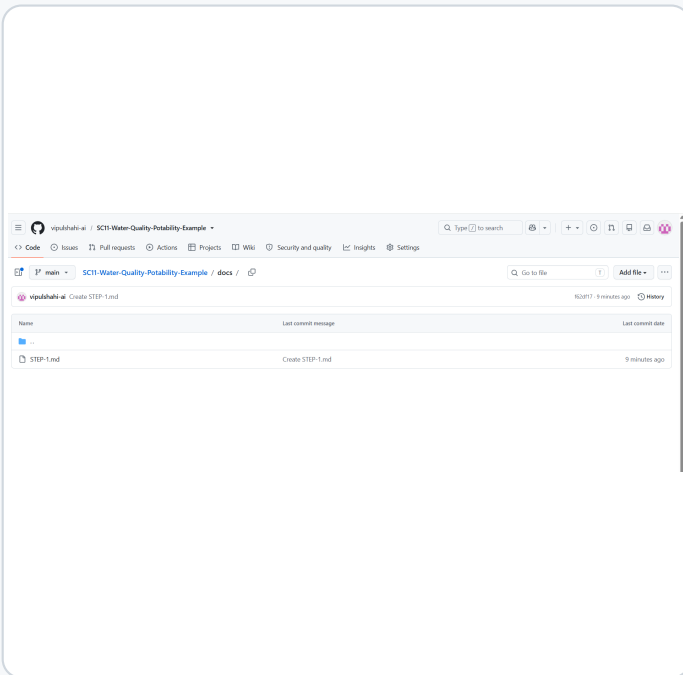
- 1 Populate docs/STEP-1.md with the team contract and planned M1 method.
- 2 Replace the root README placeholder with the formal Data Dictionary.
- 3 Create data/sample\_input.csv with 20 inspectable rows and a deliberate missing-value test case.
- 4 Create src/validate\_data.py and requirements.txt placeholders for local implementation.

*Follow the screenshots and the click path exactly. Stop and verify each checkpoint before moving forward.*

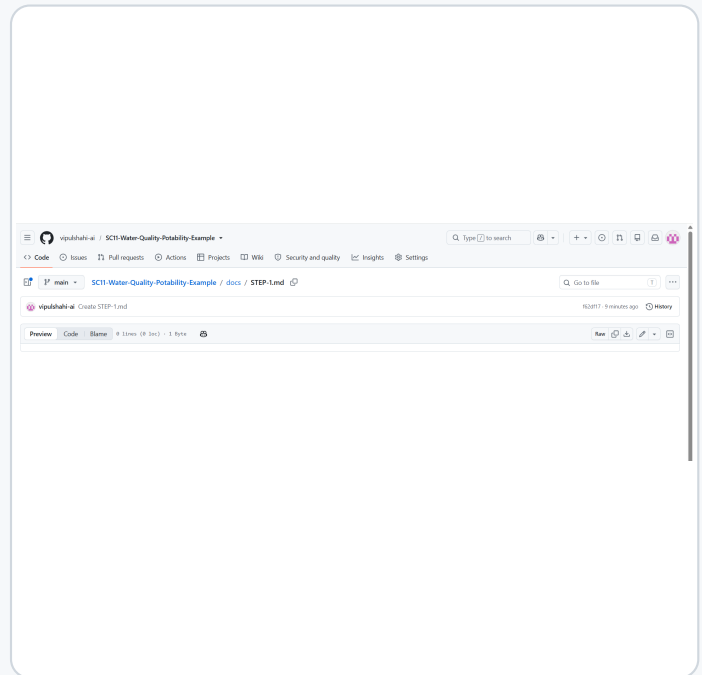
# Open STEP-1.md in Edit Mode

Use the pencil icon on the file page, not the browser address bar or repository README editor.

**WHERE TO CLICK** Repository -> docs folder -> STEP-1.md -> pencil / Edit this file icon



Screenshot 103755 - docs folder



Screenshot 103809 - STEP-1.md editor

## DO THIS - CLICK BY CLICK

- 1 Open the repository Code page.
- 2 Click docs in the file list.
- 3 Click STEP-1.md.
- 4 On the file page, locate the pencil icon / Edit this file control near the top-right of the file content.
- 5 Click the pencil icon.
- 6 Confirm you can now place a text cursor inside the Markdown editor.

### EXPECTED RESULT

STEP-1.md is open in GitHub edit mode and the content area is editable.

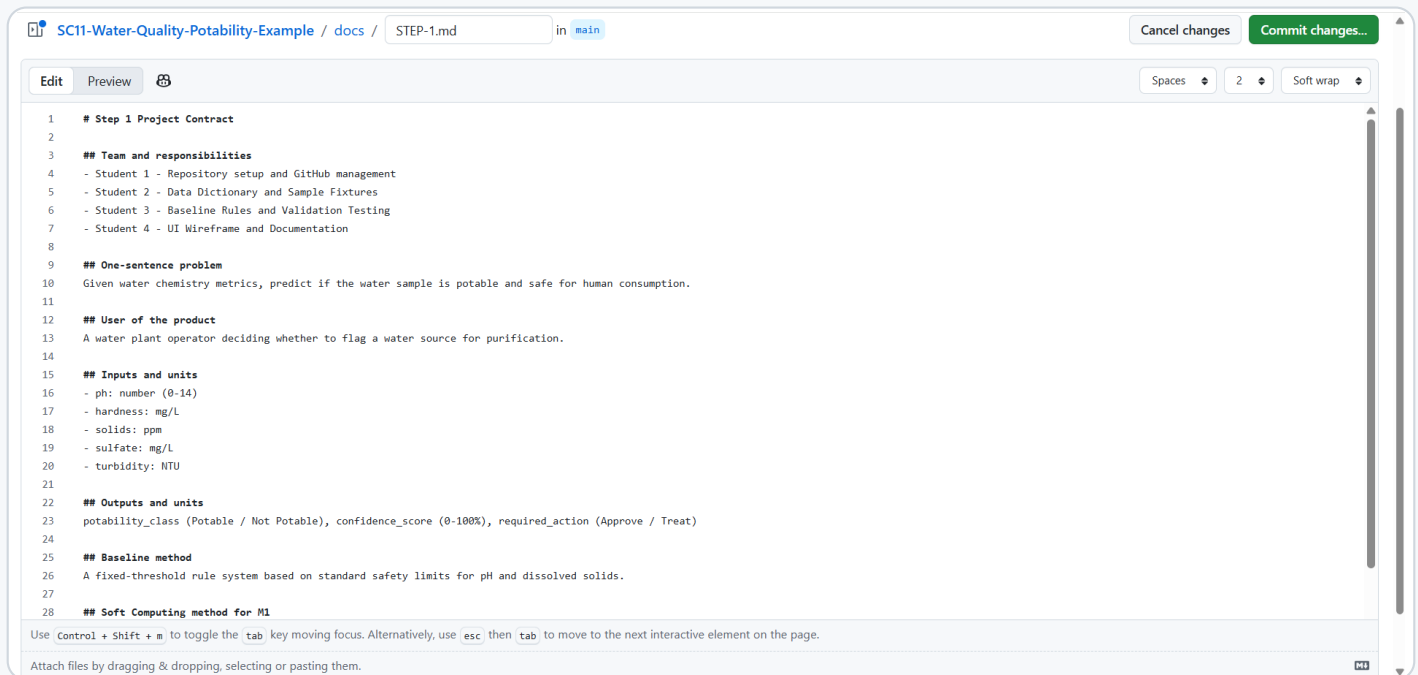
### IF YOU DO NOT SEE THIS

If you see only rendered text, you are still in Preview. Click the pencil/Edit control again.

# Paste the Project Contract

The contract should be specific enough that another teammate can understand what is being built without verbal explanation.

**WHERE TO CLICK** STEP-1.md edit mode -> main Markdown editor



Screenshot 103937 - populated STEP-1.md

## DO THIS - CLICK BY CLICK

- 1 Click inside the main editor area.
- 2 If the file contains only placeholder text, select it before pasting the approved contract.
- 3 Paste/type the Step-1 project contract.
- 4 Include team members/responsibilities, one-sentence problem, intended user, inputs/units, outputs, baseline method and M1 Soft Computing method.
- 5 Scroll through the editor once and check that headings use Markdown syntax consistently.
- 6 Use the Preview tab if available to quickly check formatting before committing.

### EXPECTED RESULT

The editor contains a complete Project Contract with all required sections and no obvious placeholder gaps.

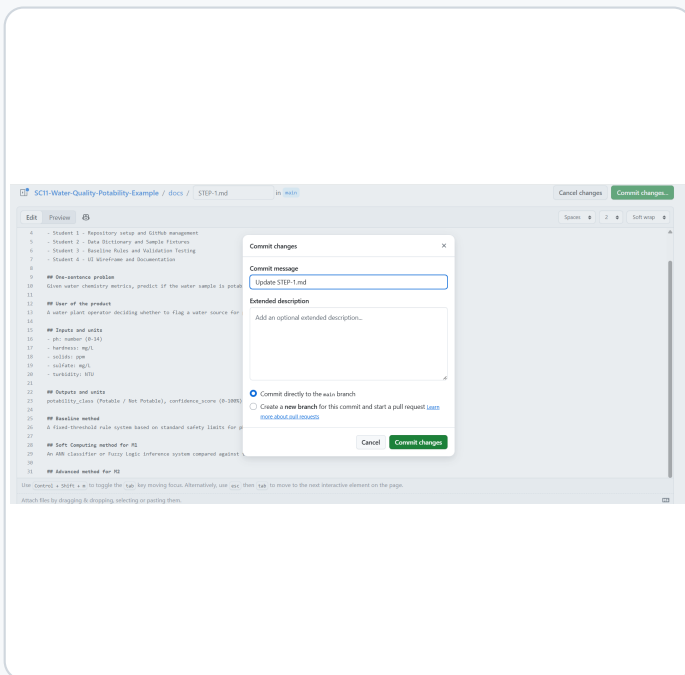
### IF YOU DO NOT SEE THIS

If Markdown headings or bullets look wrong in Preview, fix the syntax before committing. Do not use screenshots as a substitute for editable project documentation.

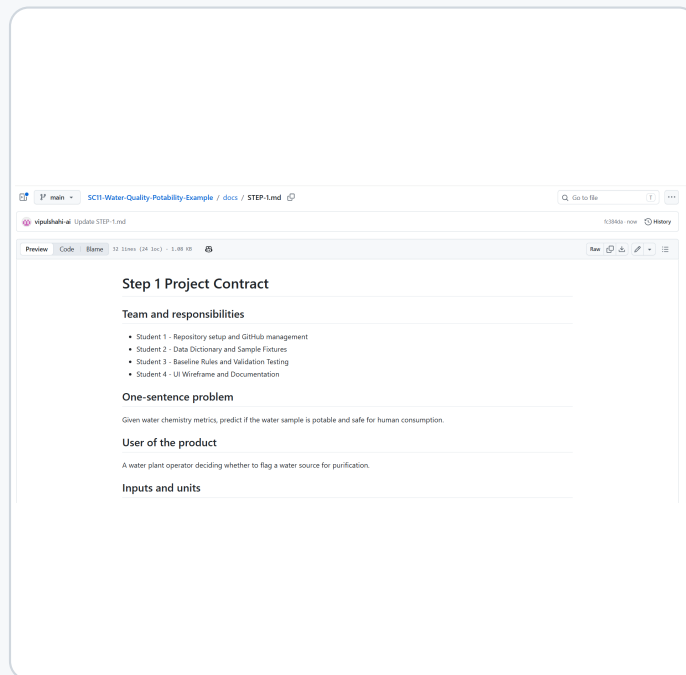
# Commit and Verify the Project Contract

A good documentation update should be visible as rendered Markdown after commit.

**WHERE TO CLICK** Editor -> Commit changes... -> descriptive message -> Commit directly to main -> Commit changes



Screenshot 103951 - commit dialog



Screenshot 104037 - rendered contract

## DO THIS - CLICK BY CLICK

- 1 Click Commit changes... at the top-right.
- 2 Enter/review a descriptive commit message such as Update STEP-1.md.
- 3 Keep Commit directly to the main branch selected.
- 4 Click the final green Commit changes button.
- 5 After GitHub returns to the file, switch to rendered/Preview view if needed.
- 6 Read the contract top-to-bottom and verify headings, roles, problem, inputs, outputs and methods are all visible.

### TYPE / SELECT EXACTLY

Update STEP-1.md

### EXPECTED RESULT

GitHub renders STEP-1.md as readable headings and bullet points.

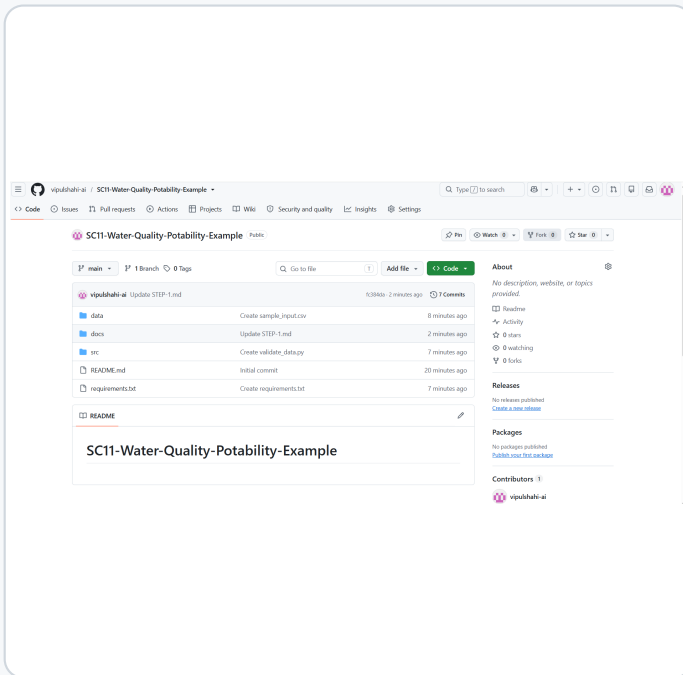
### IF YOU DO NOT SEE THIS

If the commit succeeded but the file still looks unformatted, inspect Markdown symbols (#, -, tables) rather than re-uploading the file.

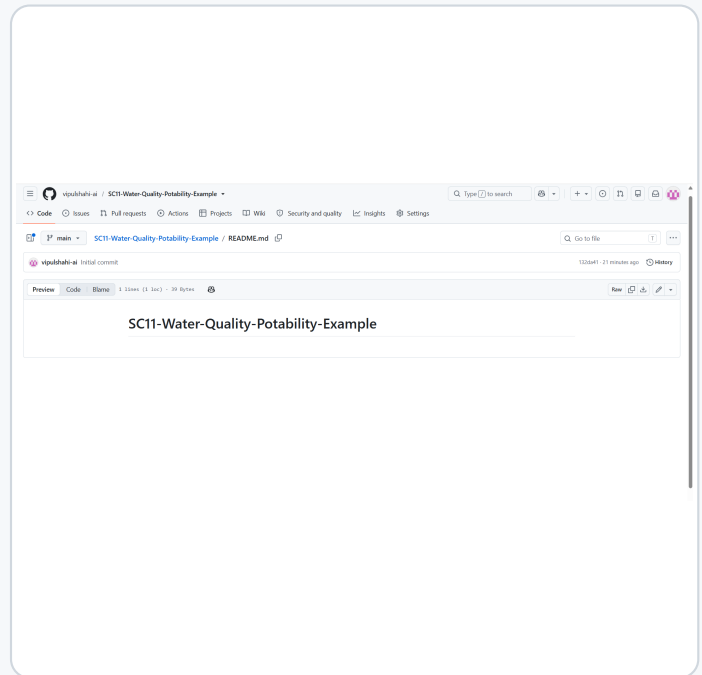
# Open the Root README for the Data Dictionary

The formal Data Dictionary in this example is stored in the root README.md, not data/README.md.

**WHERE TO CLICK** Repository root -> README.md -> pencil / Edit this file



Screenshot 104211 - repository root



Screenshot 104225 - root README

## DO THIS - CLICK BY CLICK

- 1 Click the repository name in the breadcrumb to return to root.
- 2 In the root file list, click README.md (not data/README.md).
- 3 On the README file page, click the pencil/Edit this file icon.
- 4 Place the cursor in the editor.
- 5 Replace the minimal placeholder content with the approved Data Dictionary Markdown.

### EXPECTED RESULT

The root README.md is open in edit mode and ready for the Data Dictionary.

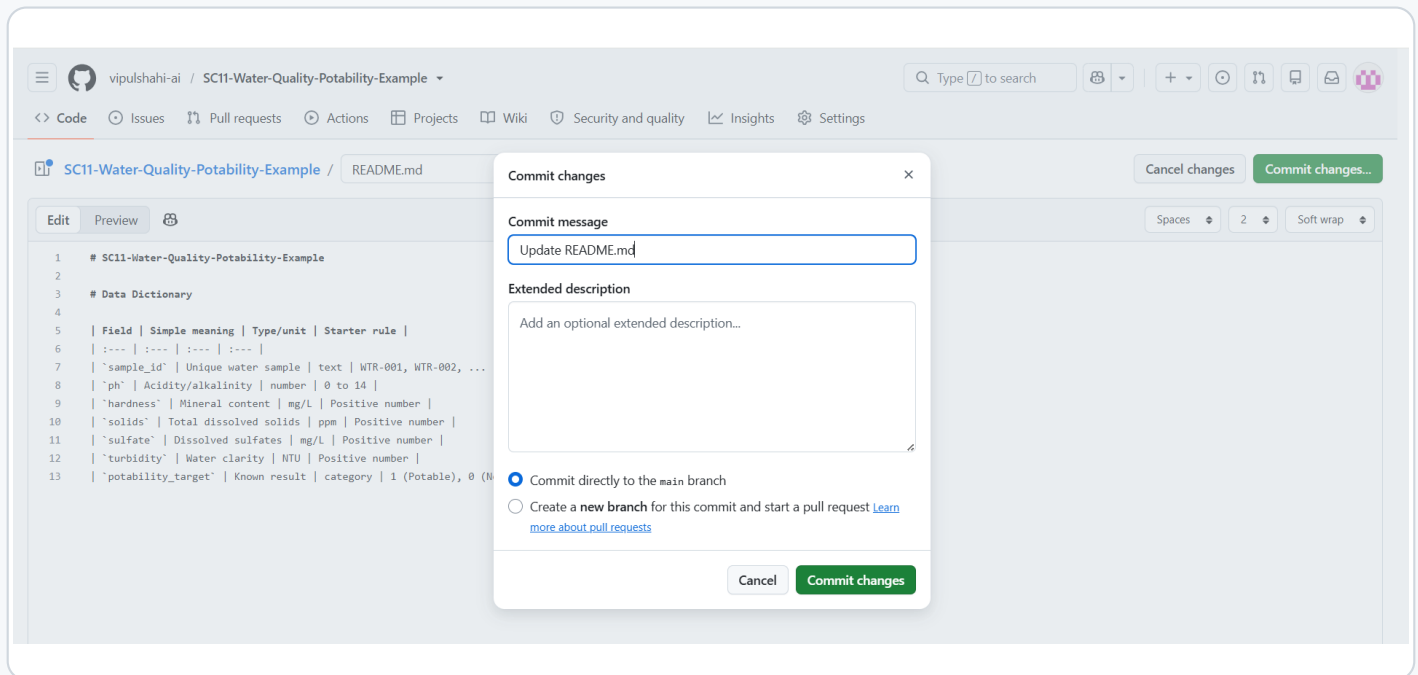
### IF YOU DO NOT SEE THIS

If the breadcrumb says data / README.md, you opened the wrong README. Return to root and open the root-level README.md.

# Commit the Data Dictionary

Define every starter field before you paste the CSV rows.

**WHERE TO CLICK** Root README edit mode -> Commit changes... -> commit dialog



Screenshot 104249 - README commit dialog

## DO THIS - CLICK BY CLICK

- 1 Confirm your Data Dictionary includes each starter field used by the dataset.
- 2 For each field, include a plain-English meaning, type/unit and starter constraint/rule.
- 3 Click Commit changes....
- 4 Use a descriptive message such as Update data dictionary.
- 5 Keep direct-to-main selected.
- 6 Click the final Commit changes button.

### TYPE / SELECT EXACTLY

Update data dictionary

### EXPECTED RESULT

The root README.md update is committed on main.

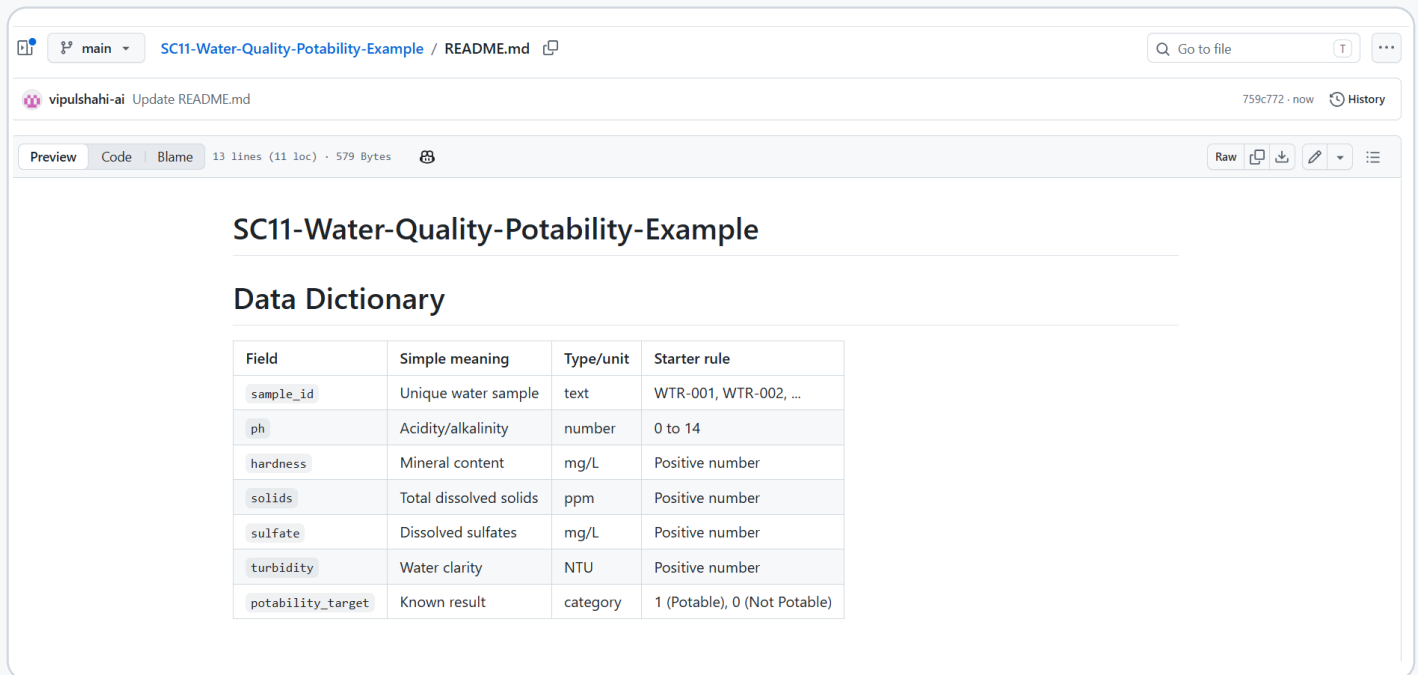
### IF YOU DO NOT SEE THIS

If the commit dialog shows a different file path, cancel and confirm you are editing the root README.md.

# Verify the Rendered Data Dictionary

The table must be readable before the data is created.

**WHERE TO CLICK** Root README.md -> rendered file view



Screenshot 104313 - rendered Data Dictionary

## DO THIS - CLICK BY CLICK

- 1 Read the rendered Data Dictionary table.
- 2 Confirm the field-name column includes sample\_id, ph, hardness, solids, sulfate, turbidity and potability\_target.
- 3 Confirm each row has a meaning and a type/unit.
- 4 Confirm target values/rules are stated clearly.
- 5 Fix any mismatch now; the CSV header in the next steps must use the same field names.

### EXPECTED RESULT

A clean rendered table is visible and every CSV field planned for Step 1 is defined.

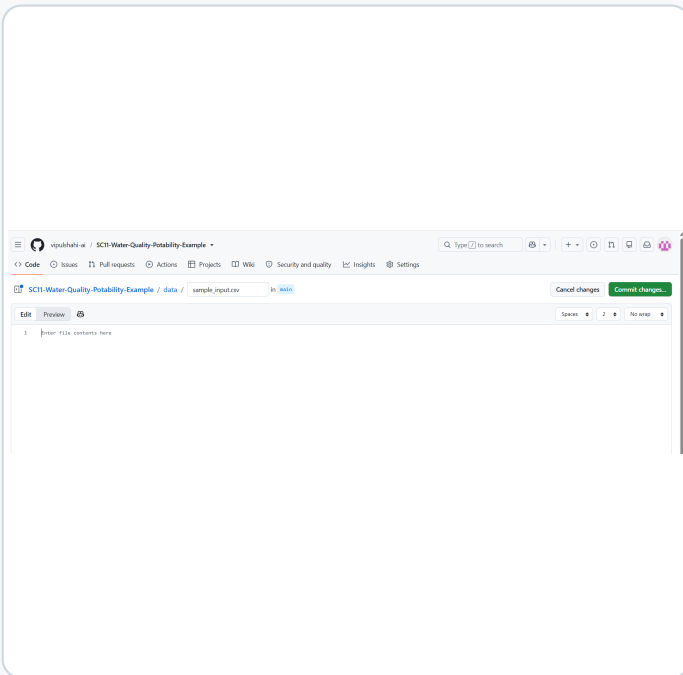
### IF YOU DO NOT SEE THIS

If the table renders as plain text, inspect the Markdown table separator row and pipe characters before continuing.

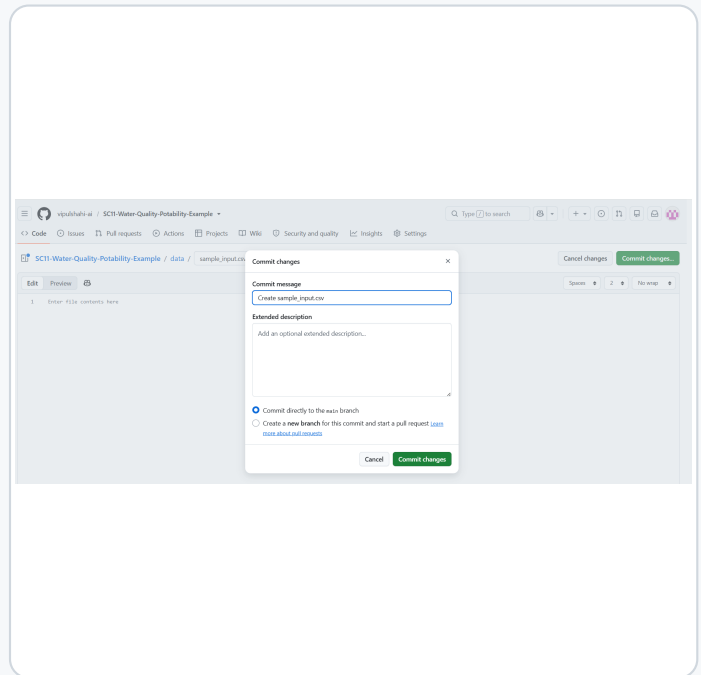
# Create data/sample\_input.csv

Create the CSV placeholder under data, then verify the path before adding rows.

**WHERE TO CLICK** Repository root -> Add file -> Create new file -> Name your file...



Screenshot 103320 - new sample\_input.csv



Screenshot 103333 - commit placeholder

## DO THIS - CLICK BY CLICK

- 1 Return to repository root.
- 2 Click Add file -> Create new file.
- 3 In Name your file..., type data/sample\_input.csv exactly.
- 4 Leave the body blank for the placeholder creation if following the screenshot sequence.
- 5 Click Commit changes....
- 6 Keep direct-to-main selected and complete the commit.

**TYPE / SELECT EXACTLY**  
**data/sample\_input.csv**

### EXPECTED RESULT

The placeholder file exists under data/.

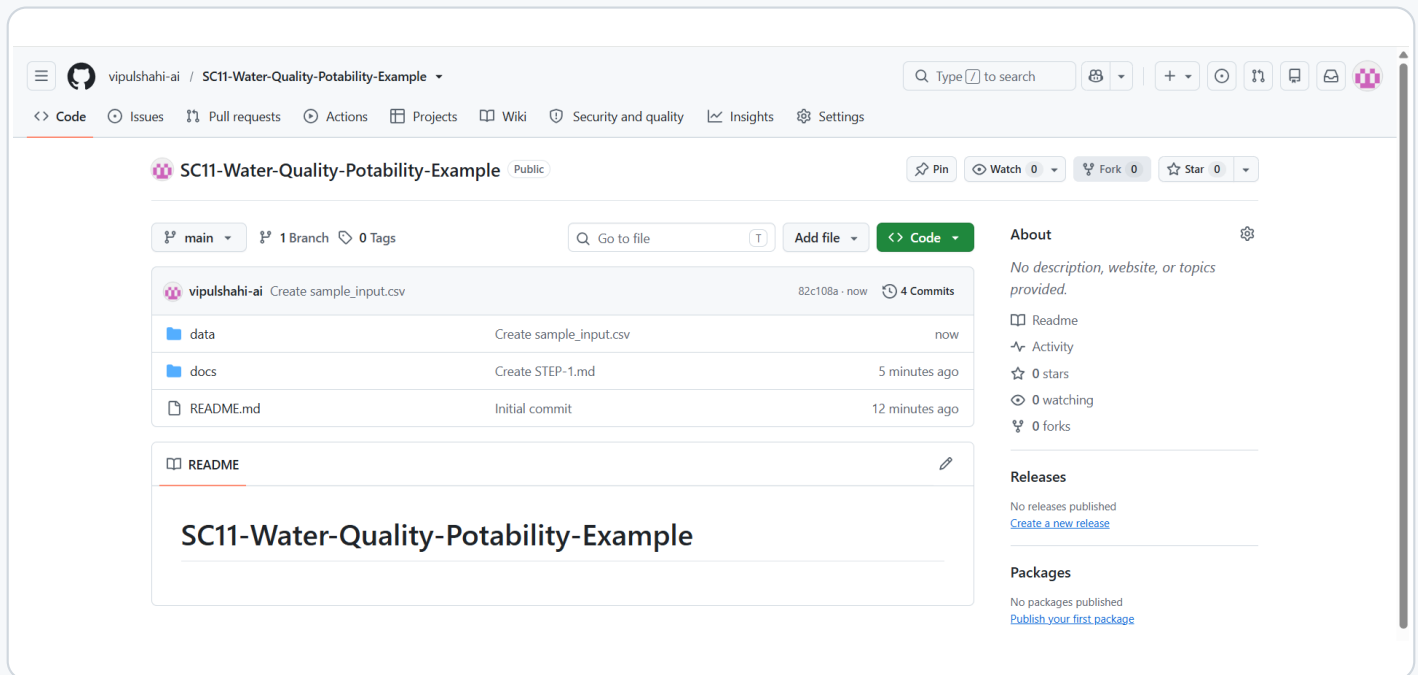
### IF YOU DO NOT SEE THIS

If sample\_input.csv appears at repository root, delete/recreate it under data/ before adding the dataset.

# Confirm sample\_input.csv Exists

Verify the placeholder before creating source files.

**WHERE TO CLICK** Repository root -> data folder -> sample\_input.csv



Screenshot 103346 - repository after CSV placeholder

## DO THIS - CLICK BY CLICK

- 1 Return to the repository root.
- 2 Open data/.
- 3 Confirm sample\_input.csv is listed next to data/README.md.
- 4 Return to root after verifying.
- 5 Do not paste the 20 rows yet if you are following the exact screenshot sequence; the source placeholders are created next.

### EXPECTED RESULT

data/sample\_input.csv exists in the correct folder.

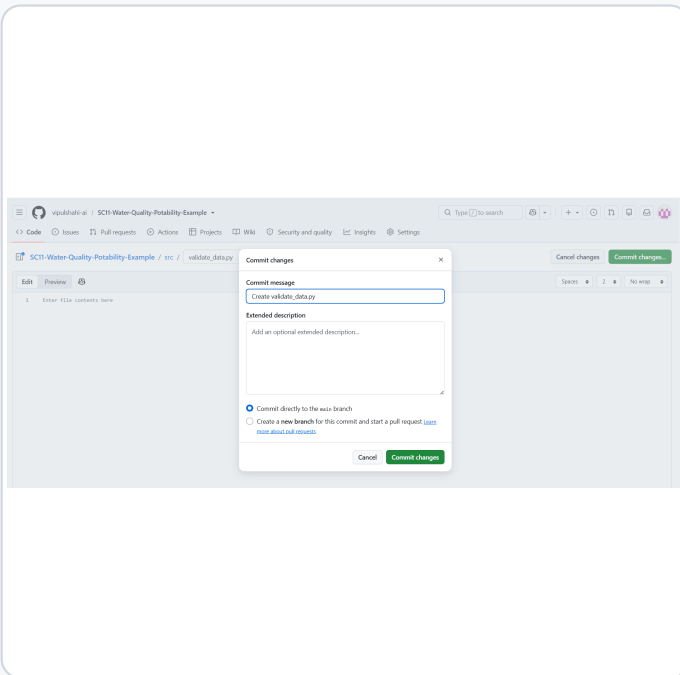
### IF YOU DO NOT SEE THIS

If the file is missing, check the latest commit and the path used during creation.

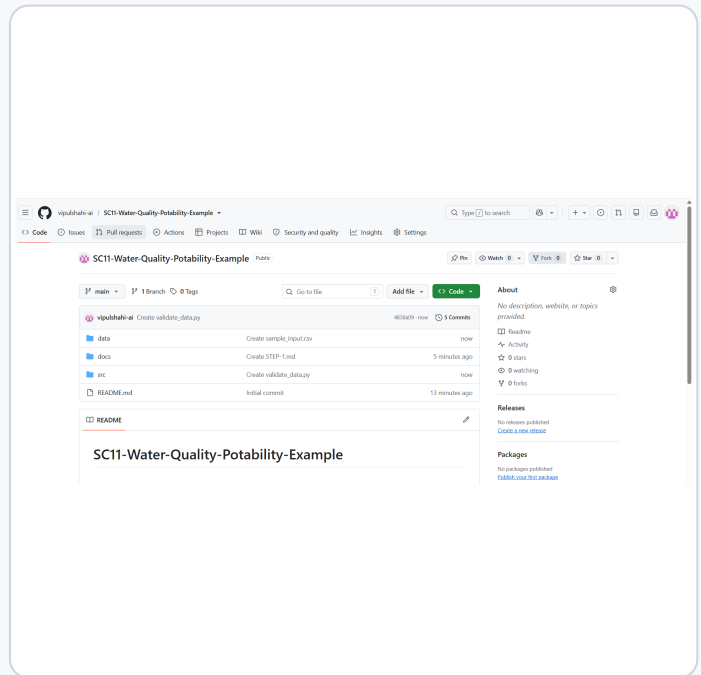
# Create src/validate\_data.py

This file will become the local Step-1 quality gate in Phase 4.

**WHERE TO CLICK** Repository root -> Add file -> Create new file -> Name your file...



Screenshot 103413 - commit validate\_data placeholder



Screenshot 103430 - src appears

## DO THIS - CLICK BY CLICK

- 1 Return to repository root.
- 2 Click Add file -> Create new file.
- 3 Type src/validate\_data.py in the filename field.
- 4 Leave the body empty/minimal as the placeholder.
- 5 Click Commit changes... and commit directly to main.
- 6 Return to root and verify a new src/ folder now exists.

**TYPE / SELECT EXACTLY**  
**src/validate\_data.py**

### EXPECTED RESULT

A src/ folder exists and contains validate\_data.py.

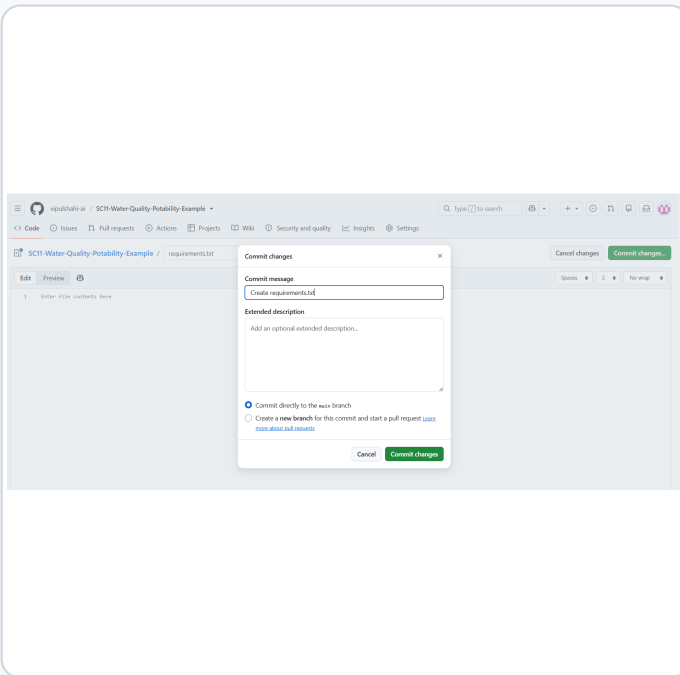
### IF YOU DO NOT SEE THIS

If validate\_data.py appears at root, the src/ prefix was omitted. Correct the path now.

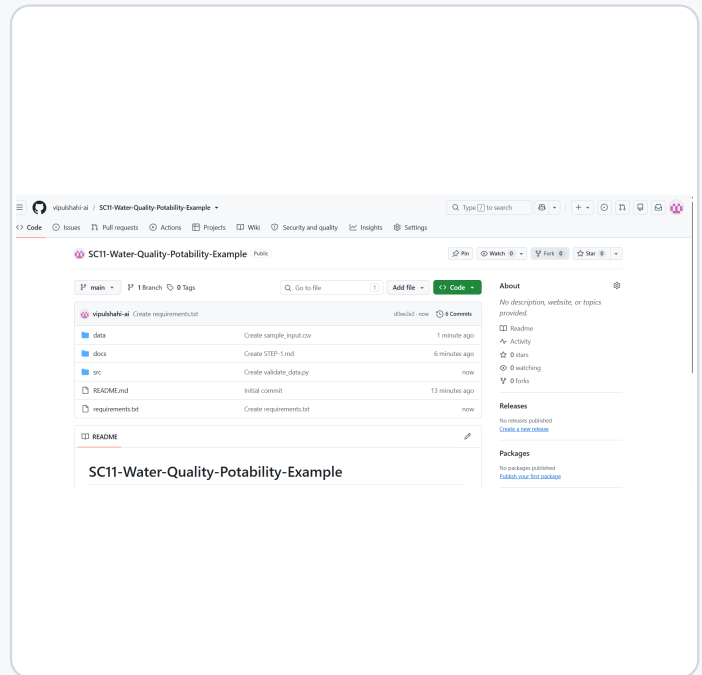
# Create requirements.txt

requirements.txt belongs at repository root, not inside src or data.

**WHERE TO CLICK** Repository root -> Add file -> Create new file -> Name your file...



Screenshot 103458 - requirements commit



Screenshot 103512 - root tree

## DO THIS - CLICK BY CLICK

- 1 Confirm the breadcrumb is at repository root.
- 2 Click Add file -> Create new file.
- 3 Type requirements.txt with no folder prefix.
- 4 Create/commit the placeholder directly to main.
- 5 Return to root and verify requirements.txt is at the same level as README.md, data/, docs/ and src/.

### TYPE / SELECT EXACTLY

**requirements.txt**

### EXPECTED RESULT

Root tree contains data/, docs/, src/, README.md and requirements.txt.

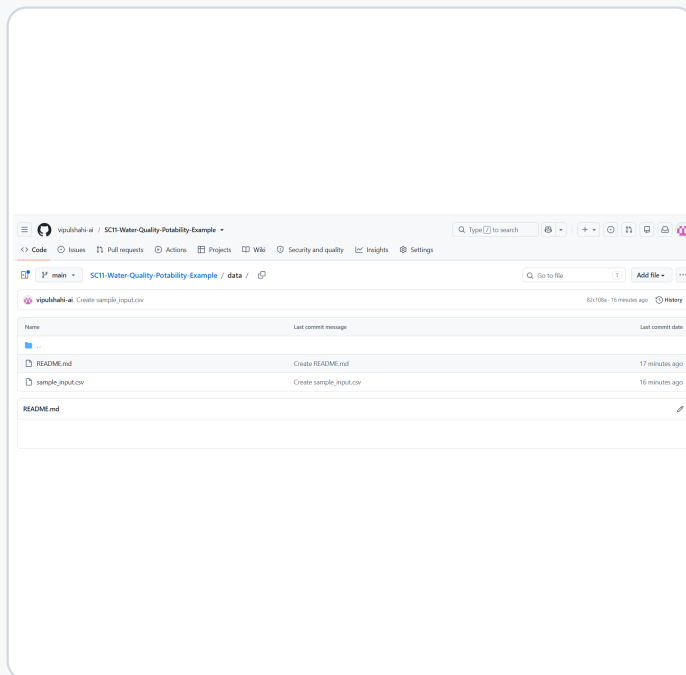
### IF YOU DO NOT SEE THIS

If requirements.txt is inside another folder, recreate it at root before moving on.

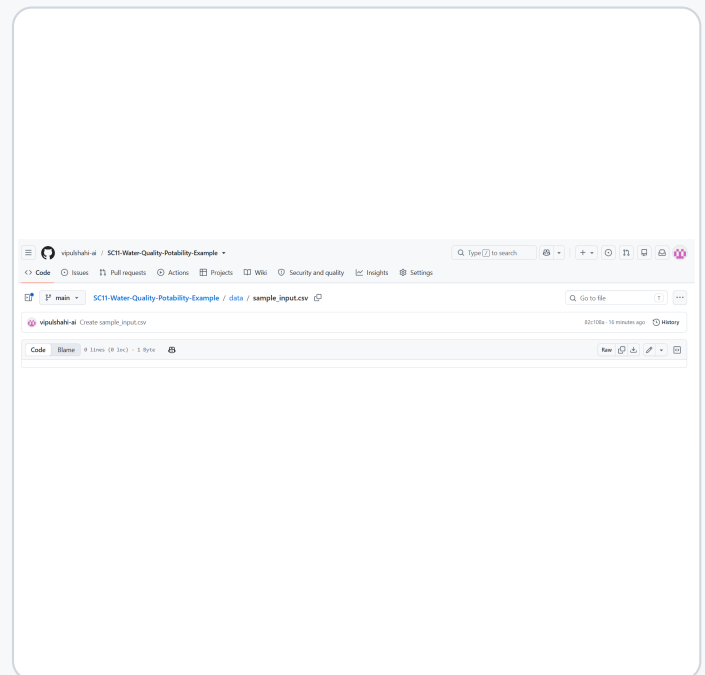
# Open sample\_input.csv for Editing

Use the file editor so the starter data is versioned immediately when committed.

**WHERE TO CLICK** Repository -> data -> sample\_input.csv -> pencil / Edit this file



Screenshot 105009 - data folder



Screenshot 105022 - CSV editor

## DO THIS - CLICK BY CLICK

- 1 Open data/ from the repository root.
- 2 Click sample\_input.csv.
- 3 Click the pencil/Edit this file.
- 4 Confirm the editor opens for sample\_input.csv.
- 5 Place the cursor on line 1 ready for the CSV header.

## EXPECTED RESULT

sample\_input.csv is open in edit mode.

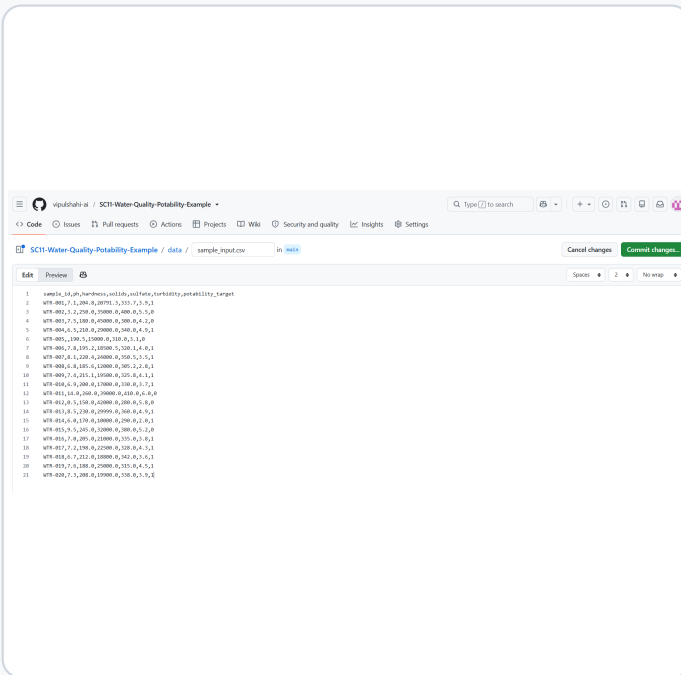
## IF YOU DO NOT SEE THIS

If GitHub shows a rendered table with no text cursor, click the pencil icon to switch to edit mode.

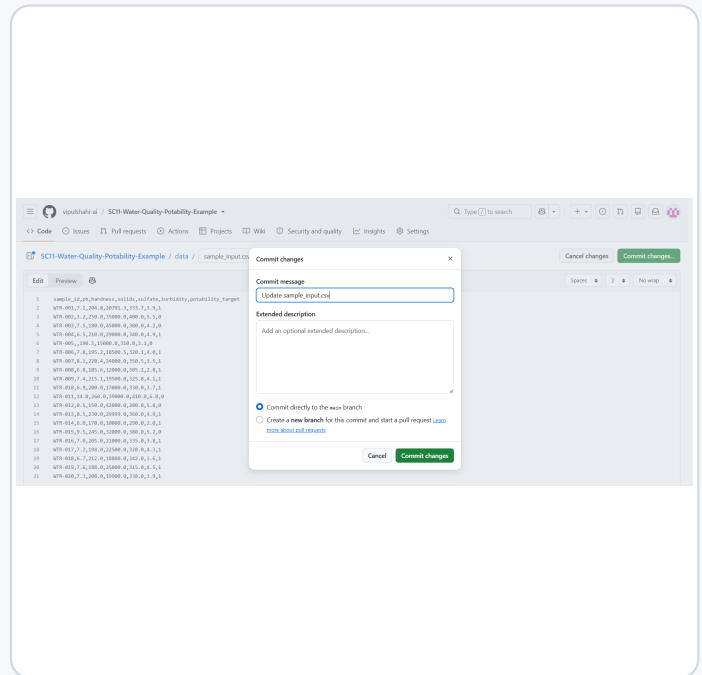
# Paste, Commit and Verify the 20-Row Starter Data

Keep the deliberate WTR-005 missing pH value at this stage; it is needed to prove the validator catches an error.

**WHERE TO CLICK** CSV editor -> paste header + 20 rows -> Commit changes...



Screenshot 105047 - pasted CSV



Screenshot 105058 - commit CSV

## DO THIS - CLICK BY CLICK

- 1 Paste the CSV header using the exact field names from the Data Dictionary.
- 2 Paste exactly 20 starter data rows for this example.
- 3 Include normal rows plus boundary/test cases.
- 4 Leave the planned WTR-005 pH field blank exactly as designed for the validation test.
- 5 Click Commit changes...
- 6 Use a descriptive message such as Update sample\_input.csv and commit directly to main.

### TYPE / SELECT EXACTLY

Update sample\_input.csv

### EXPECTED RESULT

The 20-row starter dataset is committed, including the intentional WTR-005 missing pH case.

### IF YOU DO NOT SEE THIS

If spreadsheet-like formatting changes field names or separators, return to plain-text edit mode and preserve standard comma-separated CSV syntax.

### IMPORTANT

Do not "clean" WTR-005 yet. The missing pH is intentional and will be fixed only after the validator proves it can detect the problem.

# Verify the CSV Renders Correctly

This is the last Phase-3 checkpoint before local execution.

**WHERE TO CLICK** data/sample\_input.csv -> rendered table view

The screenshot shows a GitHub file viewer interface for the file `data/sample_input.csv`. The file is rendered as a table with the following data:

|    | sample_id | ph   | hardness | solids  | sulfate | turbidity | potability_target |
|----|-----------|------|----------|---------|---------|-----------|-------------------|
| 1  | WTR-001   | 7.1  | 204.8    | 20791.3 | 333.7   | 3.9       | 1                 |
| 2  | WTR-002   | 3.2  | 250.0    | 35000.0 | 400.0   | 5.5       | 0                 |
| 3  | WTR-003   | 7.5  | 180.0    | 45000.0 | 300.0   | 4.2       | 0                 |
| 4  | WTR-004   | 6.5  | 210.0    | 29000.0 | 340.0   | 4.9       | 1                 |
| 5  | WTR-005   |      | 190.5    | 15000.0 | 310.0   | 3.1       | 0                 |
| 6  | WTR-006   | 7.8  | 195.2    | 18500.5 | 320.1   | 4.0       | 1                 |
| 7  | WTR-007   | 8.1  | 220.4    | 24000.0 | 350.5   | 3.5       | 1                 |
| 8  | WTR-008   | 6.8  | 185.6    | 12000.0 | 305.2   | 2.8       | 1                 |
| 9  | WTR-009   | 7.4  | 215.1    | 19500.0 | 325.8   | 4.1       | 1                 |
| 10 | WTR-010   | 6.9  | 200.0    | 17000.0 | 330.0   | 3.7       | 1                 |
| 11 | WTR-011   | 14.0 | 260.0    | 39000.0 | 410.0   | 6.0       | 0                 |
| 12 | WTR-012   | 0.5  | 150.0    | 42000.0 | 280.0   | 5.8       | 0                 |
| 13 | WTR-013   | 8.5  | 230.0    | 29999.0 | 360.0   | 4.9       | 1                 |

Screenshot 105126 - rendered CSV table

## DO THIS - CLICK BY CLICK

- 1 Open the committed `sample_input.csv` file.
- 2 Confirm GitHub renders columns for all seven fields.
- 3 Count/scan the sample IDs and confirm the intended 20 rows are present.
- 4 Locate WTR-005 and verify the pH cell is intentionally blank.
- 5 Confirm the header names match the Data Dictionary exactly.

### EXPECTED RESULT

The CSV renders as a table, field names match the Data Dictionary and WTR-005 shows the planned missing pH.

### IF YOU DO NOT SEE THIS

If columns collapse into one field, inspect commas and line breaks in the raw file before proceeding.

# Baseline Logic & Local Data Validation

Move from browser-only setup to a local VS Code workspace. Document a simple baseline, clone the repository, implement validation and prove that the deliberate bad row can be detected and corrected.

**By the end of this phase, students should be able to:**

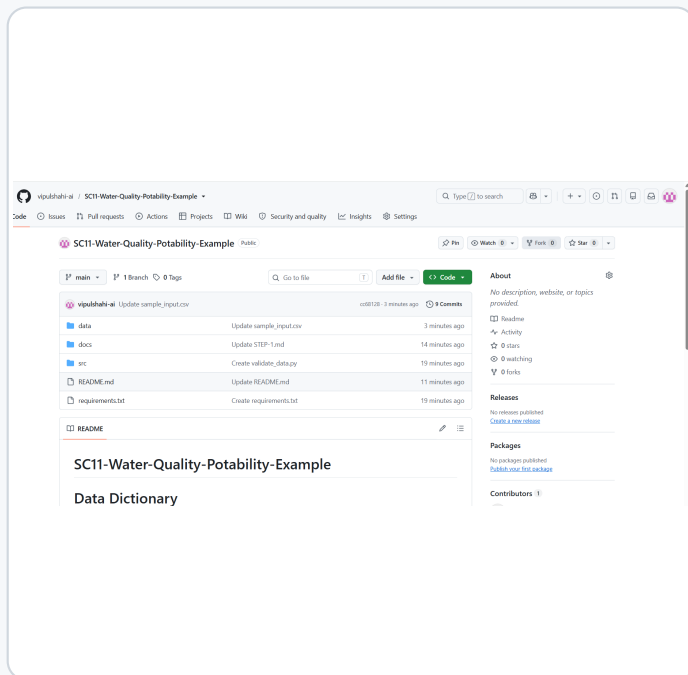
- 1 Create and commit docs/baseline-pseudocode.md.
- 2 Clone the shared repository into VS Code and trust only the known team folder.
- 3 Populate src/validate\_data.py with executable checks.
- 4 Run the validator, observe the intentional WTR-005 failure, fix the data and obtain STEP 1 DATA CHECK PASSED.

*Follow the screenshots and the click path exactly. Stop and verify each checkpoint before moving forward.*

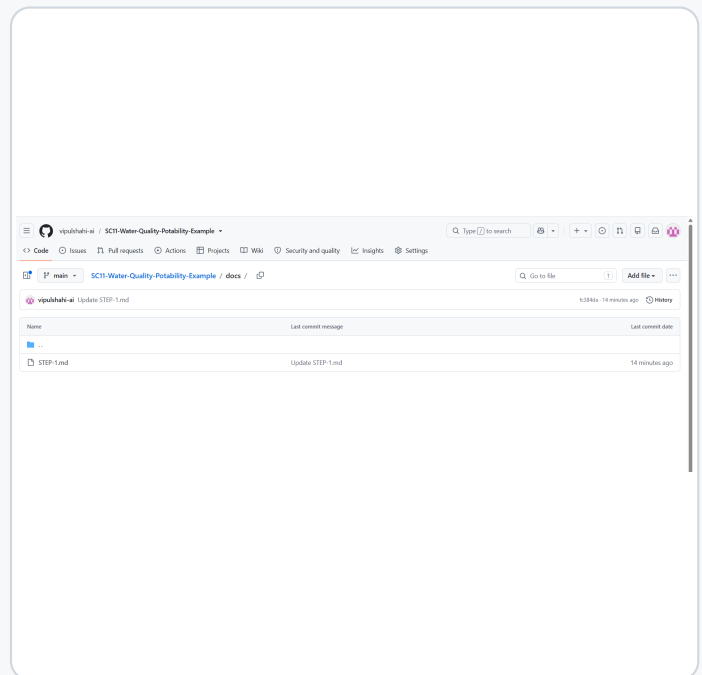
# Navigate to docs and Start a New File

Create the baseline beside STEP-1.md, not at repository root.

**WHERE TO CLICK** Repository root -> docs folder -> Add file -> Create new file



Screenshot 105414 - repository root



Screenshot 105430 - docs folder

## DO THIS - CLICK BY CLICK

- 1 Open the repository Code page.
- 2 Click docs/.
- 3 Confirm STEP-1.md is visible inside docs.
- 4 Above the folder file list, click Add file.
- 5 Choose Create new file.

### EXPECTED RESULT

GitHub opens the new-file editor while the breadcrumb is already inside docs/.

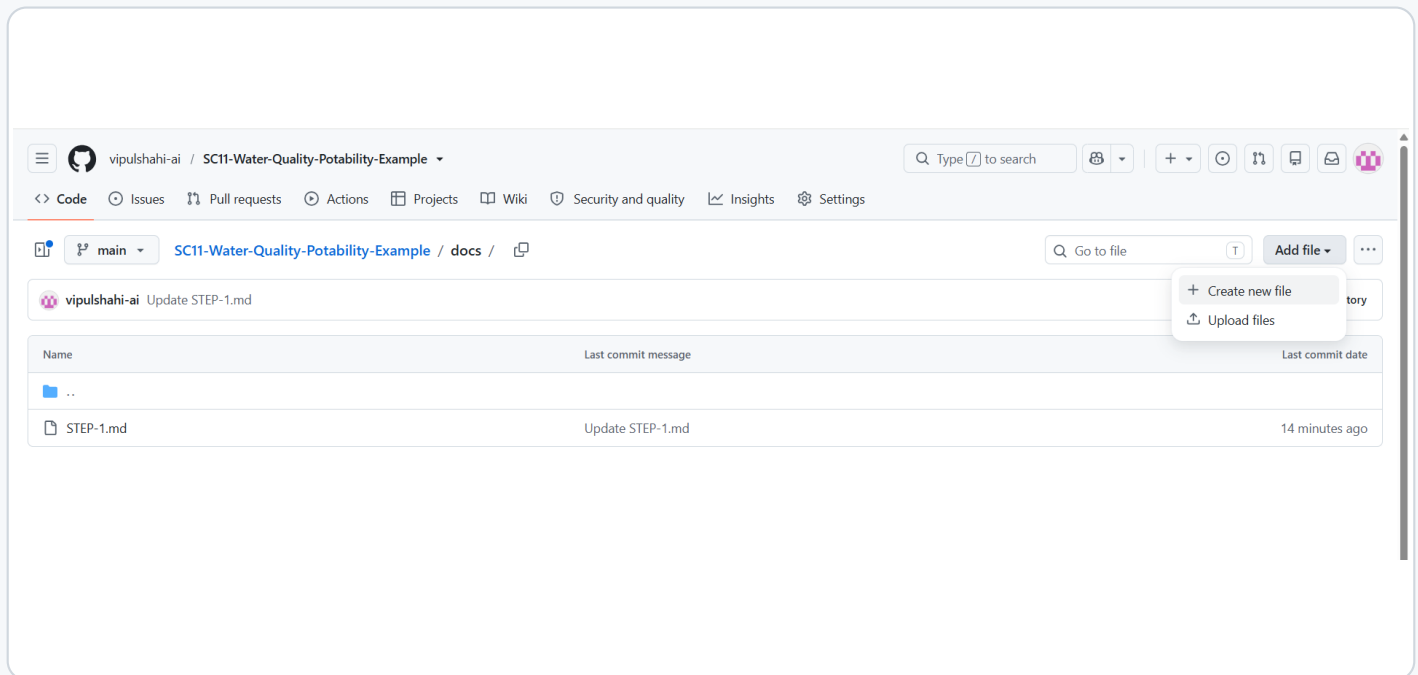
### IF YOU DO NOT SEE THIS

If the breadcrumb is at repository root, you can still use docs/baseline-pseudocode.md as the full path; however, following the screenshots, first open docs/.

# Name baseline-pseudocode.md

Because you are already inside docs, type only the filename unless the editor shows root context.

**WHERE TO CLICK** docs folder -> Create new file -> Name your file...



Screenshot 105441 - Add file / new file context

## DO THIS - CLICK BY CLICK

- 1 Click the filename field.
- 2 Type baseline-pseudocode.md.
- 3 Check the breadcrumb/path and confirm the resulting full path is docs/baseline-pseudocode.md.
- 4 Do not create a duplicate baseline file at repository root.

### TYPE / SELECT EXACTLY

**baseline-pseudocode.md**

### EXPECTED RESULT

The new file path resolves to docs/baseline-pseudocode.md.

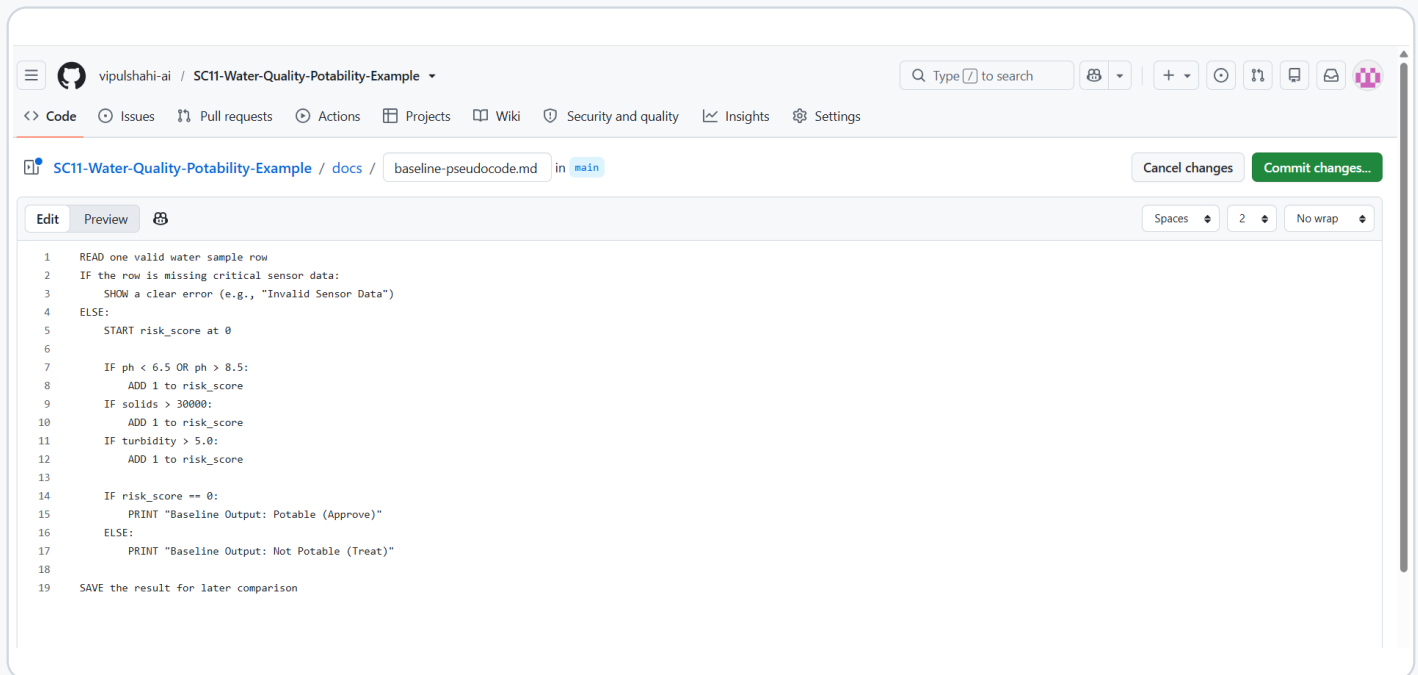
### IF YOU DO NOT SEE THIS

If the path shows baseline-pseudocode.md at root, either return to docs before creating it or use the full docs/baseline-pseudocode.md path.

# Enter the Baseline Rules

The baseline is a simple benchmark, not the final Soft Computing model.

**WHERE TO CLICK** baseline-pseudocode.md editor -> main text area



Screenshot 105512 - baseline pseudocode content

## DO THIS - CLICK BY CLICK

- 1 Click inside the Markdown editor.
- 2 Paste/type the approved baseline pseudocode.
- 3 Keep the logic readable in plain English or simple pseudocode.
- 4 Show inputs, a simple risk/decision rule and the final baseline output.
- 5 Do not present the illustrative classroom thresholds as regulatory/health standards.
- 6 Preview the Markdown if needed and correct formatting before commit.

### EXPECTED RESULT

The baseline logic can be read top-to-bottom by a teammate without requiring Python knowledge.

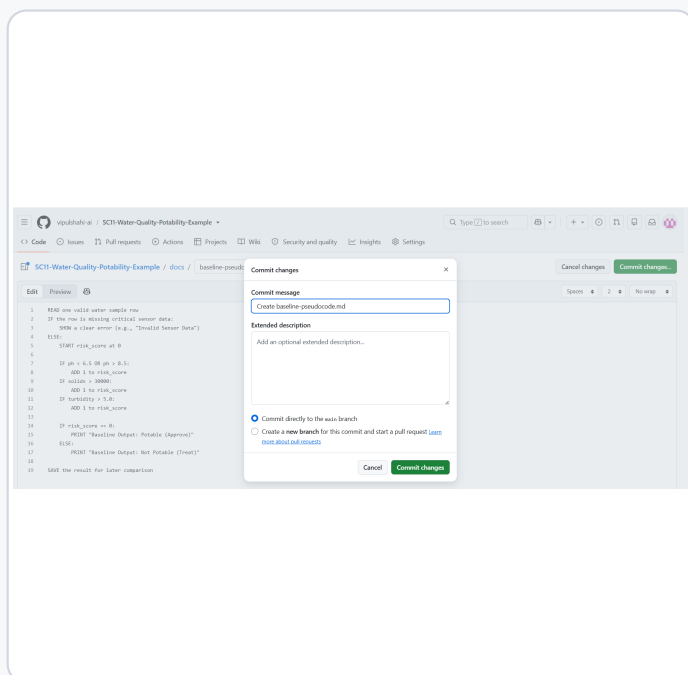
### IF YOU DO NOT SEE THIS

If the baseline starts turning into the full Soft Computing implementation, stop. Step 1 only needs a comparison benchmark plan.

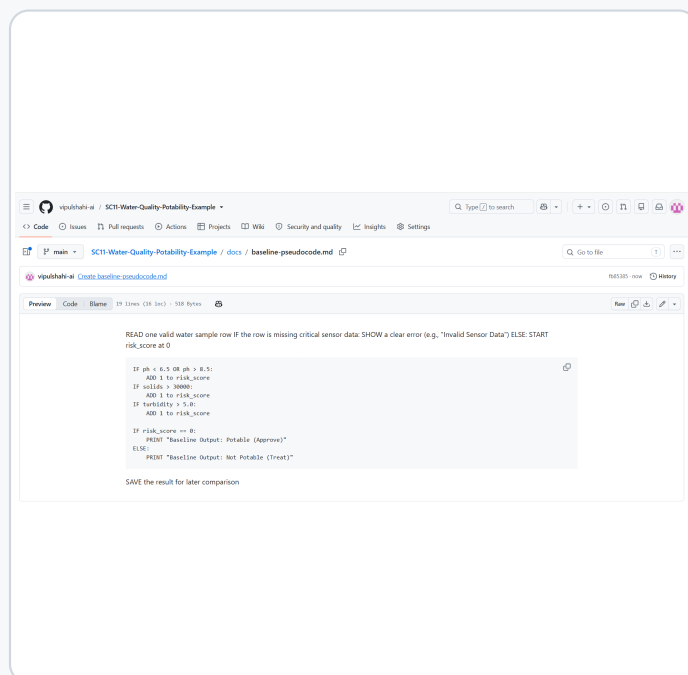
# Commit and Verify the Baseline

Commit the baseline document to main before cloning the repository.

**WHERE TO CLICK** Editor -> Commit changes... -> direct to main -> open rendered file



Screenshot 105526 - baseline commit



Screenshot 105556 - rendered baseline

## DO THIS - CLICK BY CLICK

- 1 Click Commit changes...
- 2 Enter/review a descriptive commit message such as Create baseline-pseudocode.md.
- 3 Keep direct-to-main selected and commit.
- 4 Open docs/baseline-pseudocode.md after the commit.
- 5 Confirm the rendered file shows the complete baseline logic.
- 6 If needed, use the docs folder view to confirm both STEP-1.md and baseline-pseudocode.md are present.

### TYPE / SELECT EXACTLY

Create baseline-pseudocode.md

### EXPECTED RESULT

docs/baseline-pseudocode.md is committed and renders correctly in GitHub.

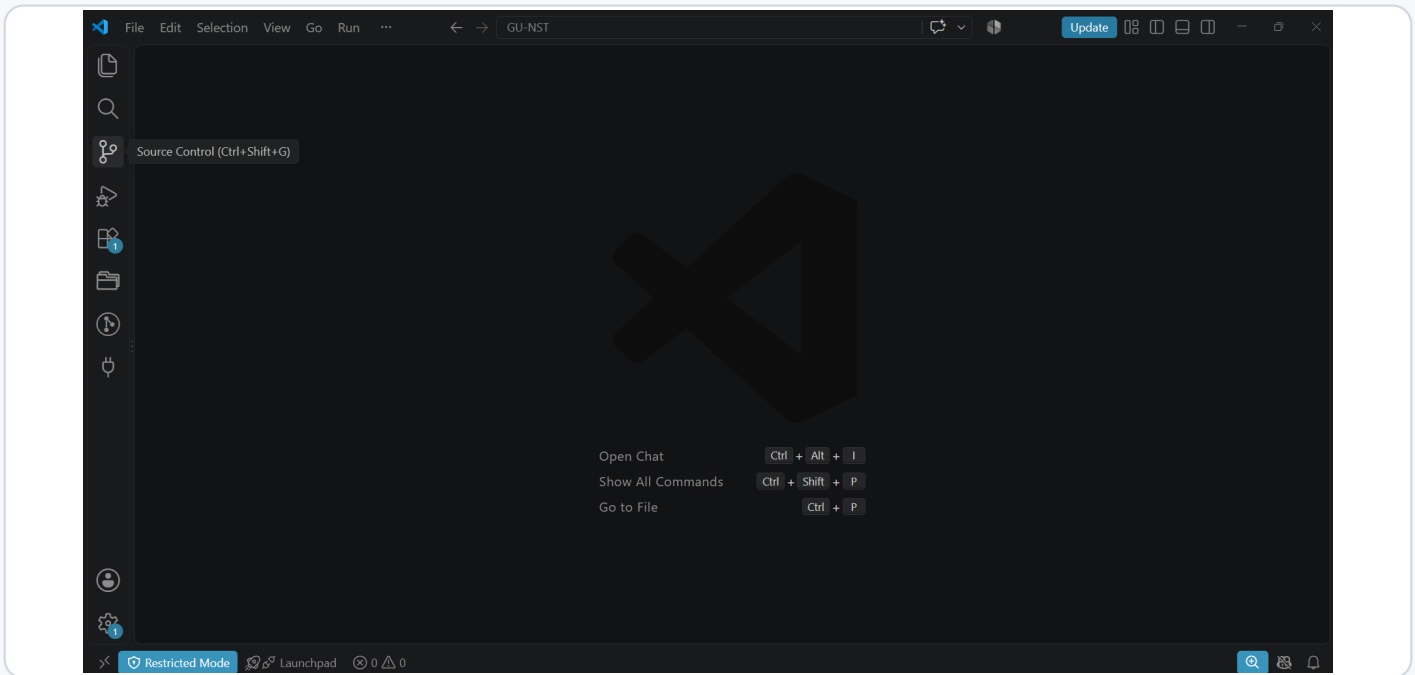
### IF YOU DO NOT SEE THIS

If the file is blank after commit, use Edit to add the missing content before cloning locally.

# Open Visual Studio Code

From this point, the repository will be worked on locally.

**WHERE TO CLICK** Windows/macOS application launcher -> Visual Studio Code



Screenshot 111939 - VS Code start screen

## DO THIS - CLICK BY CLICK

- 1 Open Visual Studio Code.
- 2 Wait for the Welcome/Start screen to load.
- 3 Do not manually create a second project folder with the same files. The next steps clone the GitHub repository so history and remote settings are preserved.

### EXPECTED RESULT

VS Code opens to the Start/Welcome screen.

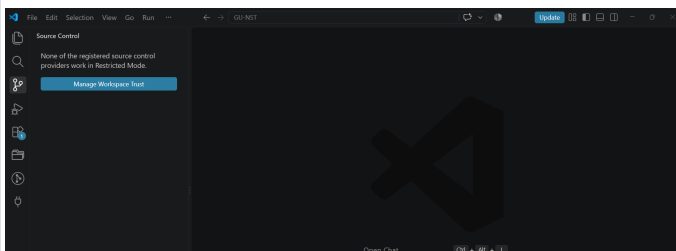
### IF YOU DO NOT SEE THIS

If a different project is already open, use File -> New Window or close the current folder before cloning the capstone repository.

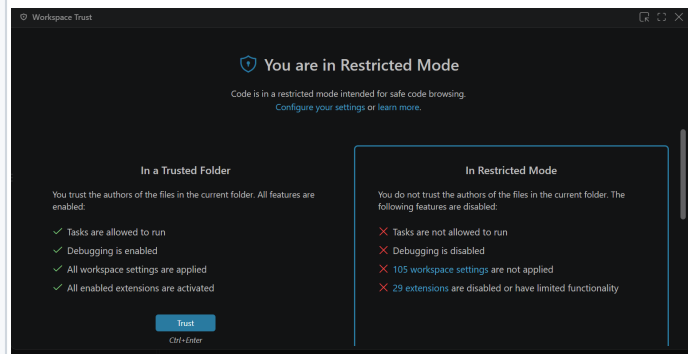
# Resolve Restricted Mode Safely

Trust only the repository created by your own team.

**WHERE TO CLICK** VS Code Restricted Mode banner / Source Control -> Manage Workspace Trust -> Trust



Screenshot 112012 - Manage Workspace Trust



Screenshot 112023 - Workspace Trust dialog

## DO THIS - CLICK BY CLICK

- 1 If VS Code shows Restricted Mode, locate the Manage Workspace Trust button/banner.
- 2 Click Manage Workspace Trust.
- 3 Read the Workspace Trust dialog.
- 4 Confirm the folder/repository is your own known capstone project.
- 5 Click Trust (or equivalent Trust this folder/workspace option).
- 6 Wait for VS Code to enable source-control and execution features.

### EXPECTED RESULT

Restricted Mode warning disappears for the trusted team repository and normal Git/source-control features are enabled.

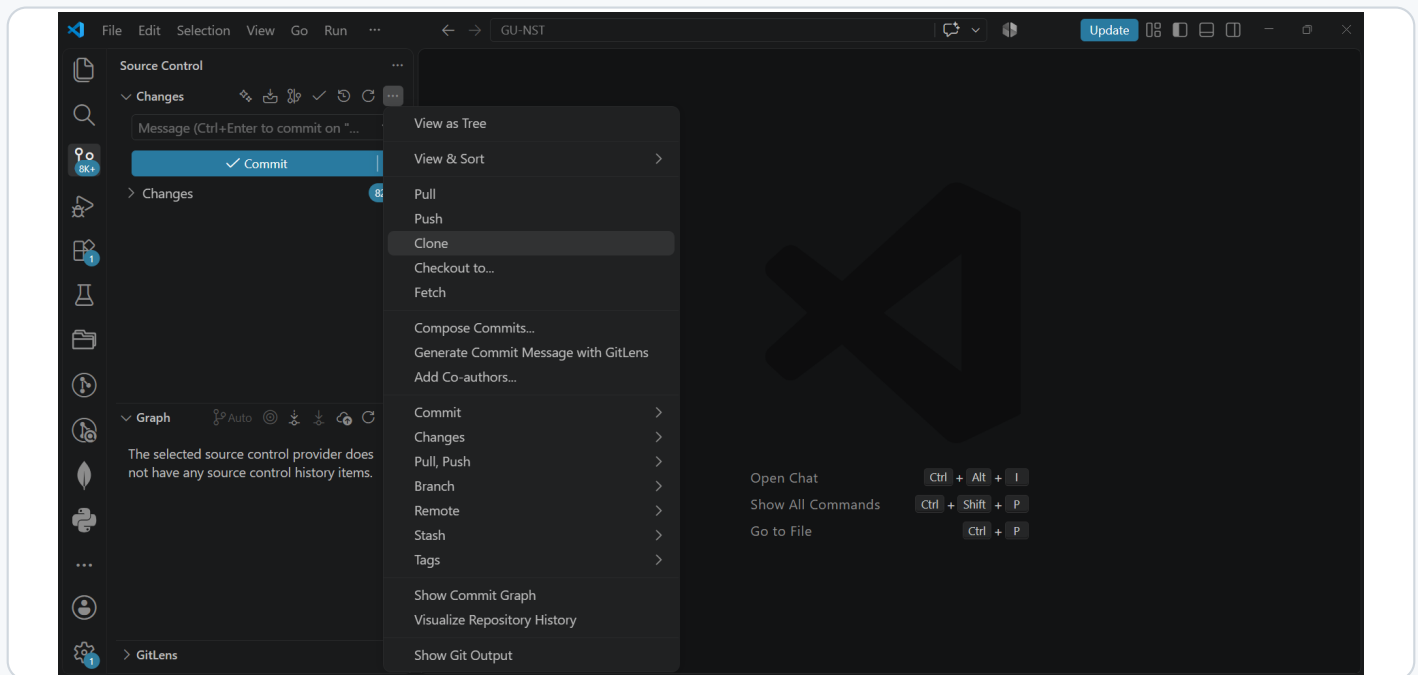
### IF YOU DO NOT SEE THIS

If the folder is not your team repository or came from an unknown source, do not trust it. Verify the folder path first.

# Open the Clone Command

Use Source Control so VS Code configures the Git remote automatically.

**WHERE TO CLICK** VS Code left Activity Bar -> Source Control icon -> ... menu -> Clone



Screenshot 112209 - Source Control menu / Clone

## DO THIS - CLICK BY CLICK

- 1 On the far-left Activity Bar, click the Source Control icon (branch/fork symbol).
- 2 At the top of the Source Control view, click the three-dot ... menu if Clone is not already offered.
- 3 Choose Clone from the menu.
- 4 Wait for the command input at the top of VS Code.

### EXPECTED RESULT

VS Code prompts for a repository URL or offers Clone from GitHub.

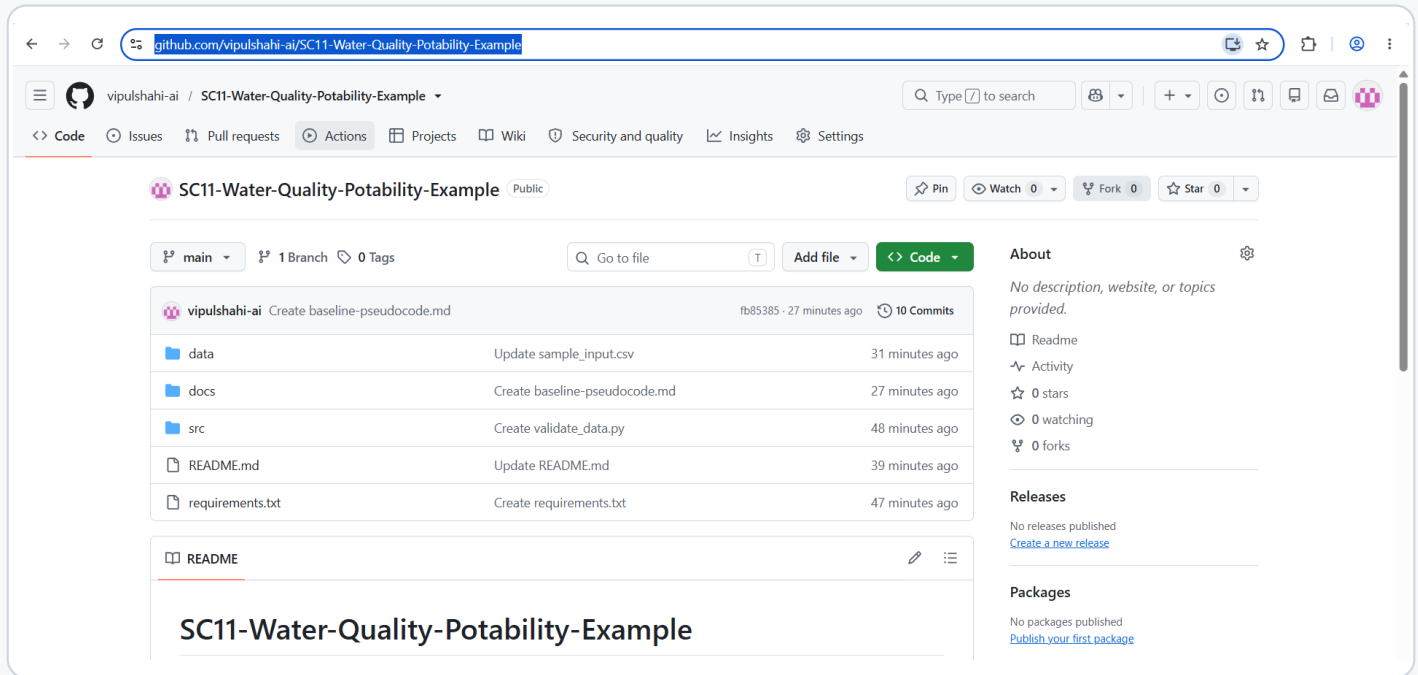
### IF YOU DO NOT SEE THIS

If Source Control is disabled, resolve Workspace Trust first. If Git is not installed/configured, VS Code may ask you to install Git.

# Copy the Repository URL

Copy the team repository address from its main GitHub page.

**WHERE TO CLICK** Web browser -> team repository root -> browser address bar (or Code -> HTTPS)



Screenshot 112302 - repository URL selected

## DO THIS - CLICK BY CLICK

- 1 Return to the GitHub repository in your browser.
- 2 Make sure you are on the repository root page, not a single file or Settings page.
- 3 Click the browser address bar and copy the full repository URL, or use Code -> HTTPS copy if preferred.
- 4 Return to VS Code after copying.

### EXPECTED RESULT

The clipboard contains the HTTPS URL for the team repository.

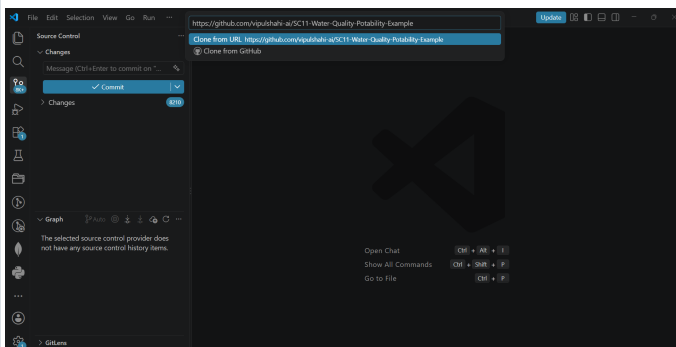
### IF YOU DO NOT SEE THIS

Do not copy a URL that points to /settings, /blob/... or /tree/... if avoidable. Use the base repository URL.

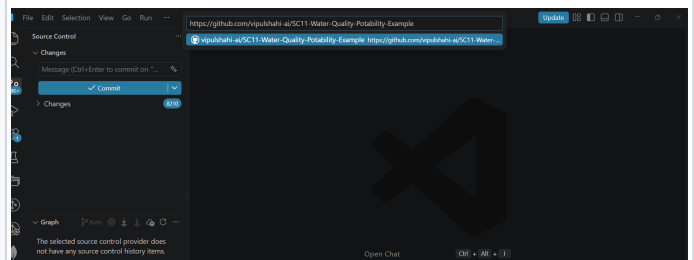
# Paste the URL into VS Code

Let VS Code recognize the remote repository.

**WHERE TO CLICK** VS Code Clone command input at top -> paste URL -> Enter



Screenshot 112318 - Clone from URL input



Screenshot 112345 - selected repository

## DO THIS - CLICK BY CLICK

- 1 Click the clone URL input at the top of VS Code if it is not already focused.
- 2 Paste the copied GitHub repository URL.
- 3 Press Enter.
- 4 If VS Code presents a matching repository suggestion, choose the intended team repository.
- 5 Wait for the local destination folder picker.

### EXPECTED RESULT

A system folder picker opens asking where the repository should be cloned.

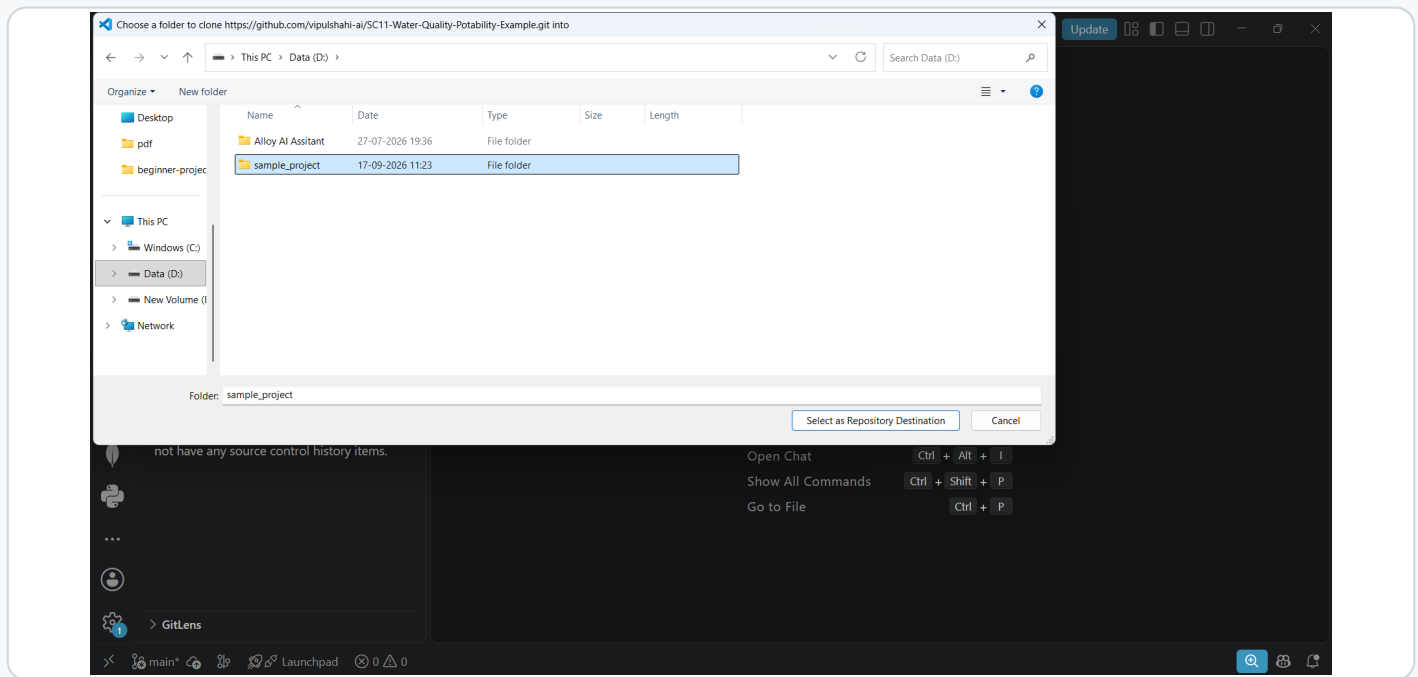
### IF YOU DO NOT SEE THIS

If authentication is requested at this stage, sign in with the GitHub account that accepted the collaborator invitation.

# Choose a Local Parent Folder

Select where the cloned repository folder should be created.

**WHERE TO CLICK** Folder picker -> choose normal workspace parent folder -> Select Repository Location



Screenshot 112540 - Windows folder picker

## DO THIS - CLICK BY CLICK

- 1 Browse to a normal workspace location such as Documents/Projects or another approved project directory.
- 2 Select the parent folder in which VS Code should create the cloned repository folder.
- 3 Avoid system folders such as Program Files or other protected locations.
- 4 Click the folder-selection confirmation button.
- 5 Wait for the clone operation to complete.

### EXPECTED RESULT

VS Code finishes cloning and prompts you to open the cloned repository.

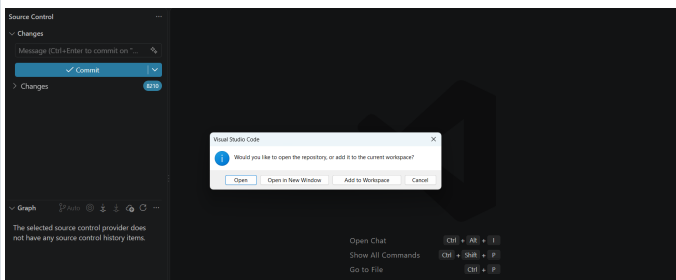
### IF YOU DO NOT SEE THIS

If Windows reports permission problems, choose a writable user folder such as Documents instead of a system directory.

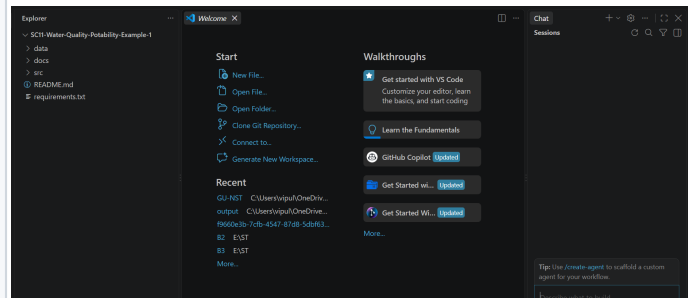
# Open the Cloned Repository

Open the new clone as the active VS Code workspace.

**WHERE TO CLICK** VS Code clone-complete prompt -> Open / Open in New Window



Screenshot 112558 - Open cloned repo prompt



Screenshot 112614 - cloned repo open

## DO THIS - CLICK BY CLICK

- 1 When VS Code reports cloning completed, click Open or Open in New Window.
- 2 If a Workspace Trust prompt appears for the cloned folder, verify it is your team repository and trust it.
- 3 Wait for the Explorer to populate.
- 4 Check the window/folder name and make sure it matches the repository you cloned.

## EXPECTED RESULT

The cloned team repository is open as the active VS Code workspace.

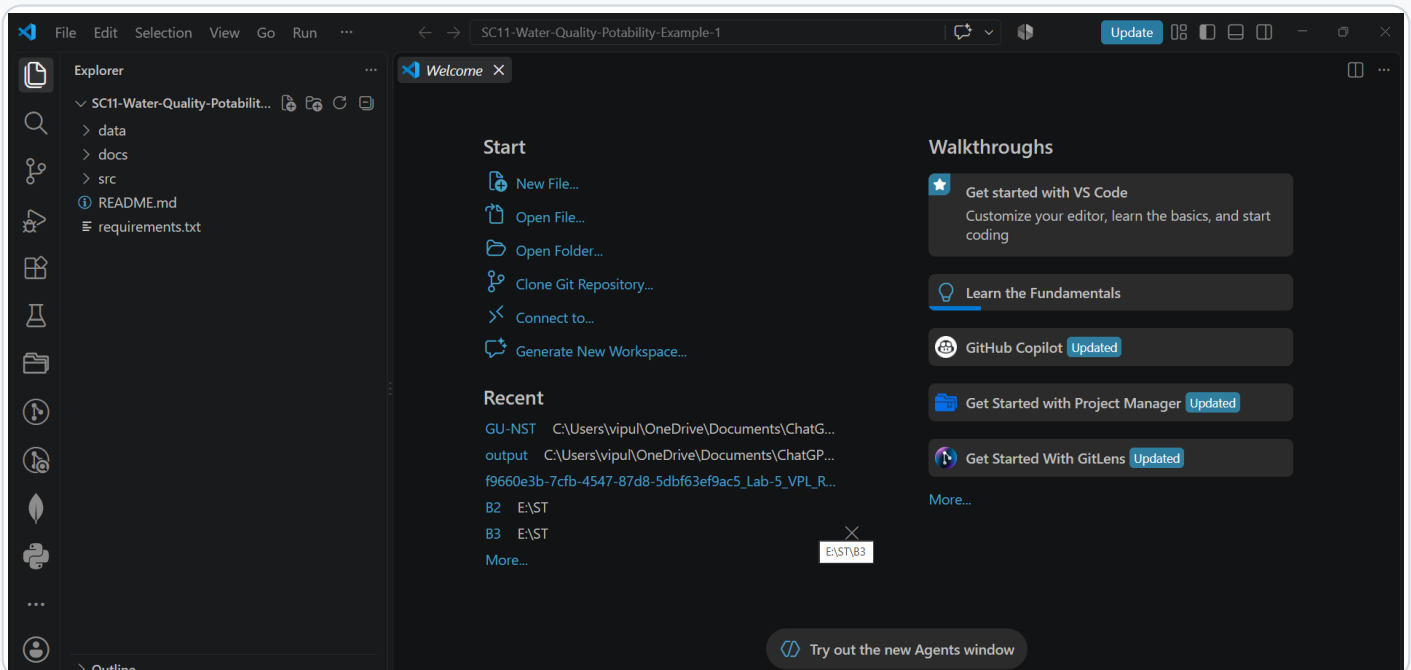
## IF YOU DO NOT SEE THIS

If you opened only the parent directory, use File -> Open Folder and choose the actual cloned repository folder containing .git and README.md.

# Verify the Local Project Tree

Do not edit code until the local clone matches GitHub.

**WHERE TO CLICK** VS Code left Activity Bar -> Explorer -> expand repository folders



Screenshot 112636 - VS Code workspace

## DO THIS - CLICK BY CLICK

- 1 Click the Explorer icon at the top of the left Activity Bar.
- 2 Expand data/, docs/ and src/.
- 3 Confirm README.md and requirements.txt appear at root.
- 4 Confirm data/sample\_input.csv exists.
- 5 Confirm docs/STEP-1.md and docs/baseline-pseudocode.md exist.
- 6 Confirm src/validate\_data.py exists.
- 7 If files are missing, do not manually recreate them yet; first verify you cloned the correct repository/branch.

## EXPECTED RESULT

The local Explorer tree matches the GitHub repository structure.

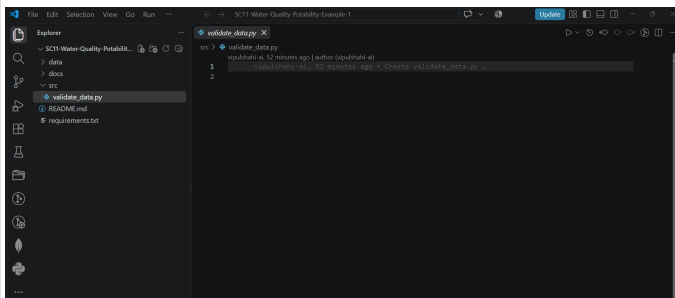
## IF YOU DO NOT SEE THIS

If the Explorer is empty, you may have opened the wrong folder. Use File -> Open Folder and select the cloned repository itself.

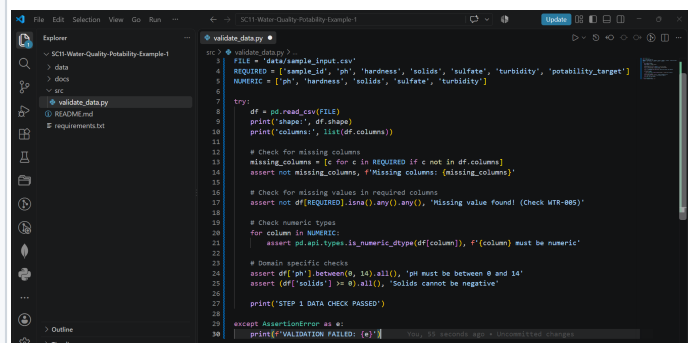
# Open and Populate validate\_data.py

Turn the placeholder into executable Step-1 validation logic.

**WHERE TO CLICK** Explorer -> src -> validate\_data.py -> editor



Screenshot 112707 - empty validator



Screenshot 112828 - validator code

## DO THIS - CLICK BY CLICK

- 1 In Explorer, expand src/.
- 2 Click validate\_data.py.
- 3 Click inside the editor and add the approved validation script.
- 4 Make sure the script reads data/sample\_input.csv using the correct relative path.
- 5 Add checks for required columns, missing values, numeric columns and simple domain constraints.
- 6 Save the file with Ctrl+S (Windows/Linux) or Cmd+S (macOS).
- 7 Confirm the editor tab no longer shows an unsaved indicator.

## EXPECTED RESULT

validate\_data.py contains executable checks and is saved locally.

## IF YOU DO NOT SEE THIS

If Python/pandas import errors appear later, confirm Python is installed and requirements/dependencies are available. Do not remove validation checks simply to make the script pass.

# Run the Validator and Expect a Failure

The first failure is deliberate evidence that the validator can detect the planned bad row.

**WHERE TO CLICK** VS Code -> Terminal -> New Terminal (if needed) -> run script from repository root

```

File Edit Selection View Go Run ... SC11-Water-Quality-Potability-Example-1
Explorer
  SC11-Water-Quality-Po...
    > data
    > docs
    > src
      validate_data.py M
    README.md
    requirements.txt

src > validate_data.py > ...
3 FILE = 'data/sample_input.csv'
4 REQUIRED = ['sample_id', 'ph', 'hardness', 'solids', 'sulfate', 'turbidity', 'potability_target']
5 NUMERIC = ['ph', 'hardness', 'solids', 'sulfate', 'turbidity']
6
7 try:
8     df = pd.read_csv(FILE)
9     print('shape:', df.shape)
10    print('columns:', list(df.columns))
11
12    # Check for missing columns
13    missing_columns = [c for c in REQUIRED if c not in df.columns]
14    assert not missing_columns, f'Missing columns: {missing_columns}'
15
16    # Check for missing values in required columns
17    assert not df[REQUIRED].isna().any().any(), 'Missing value found! (Check WTR-005)'
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

Problems Output Terminal
Terminal
1e_project\SC11-Water-Quality-Potability-Example-1\src\validate_data.py
shape: (20, 7)
columns: ['sample_id', 'ph', 'hardness', 'solids', 'sulfate', 'turbidity', 'potability_target']
VALIDATION FAILED: Missing value found! (Check WTR-005)

D:\sample_project\SC11-Water-Quality-Potability-Example-1>
  
```

Screenshot 113834 - validator failure

## DO THIS - CLICK BY CLICK

- 1 Open the integrated terminal using Terminal -> New Terminal if it is not already visible.
- 2 Check the terminal prompt path; it should point to the repository root.
- 3 Run the validator using Python.
- 4 Read the printed shape/column output.
- 5 Confirm execution stops on the missing-value assertion for WTR-005.
- 6 Treat this as a successful validation test, not as a reason to delete the check.

### TYPE / SELECT EXACTLY

`python src/validate_data.py`

### EXPECTED RESULT

The script reports the 20 x 7 data shape and fails because the planned WTR-005 pH value is missing.

### IF YOU DO NOT SEE THIS

If Python says the file cannot be found, confirm the terminal is at repository root and the script path is `src/validate_data.py`.

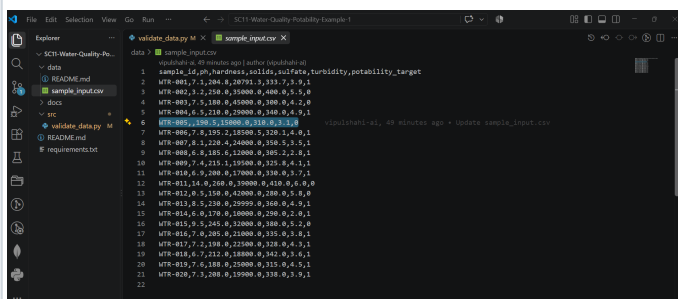
### IMPORTANT

Do not weaken or comment out the missing-value check. The correct next action is to repair the known bad input row.

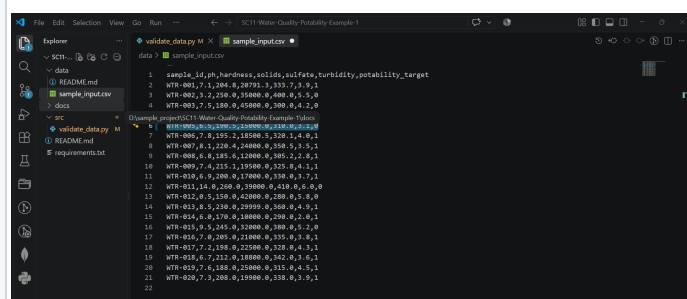
# Repair WTR-005 in the CSV

Fix the data only after the validator has proved it can catch the issue.

**WHERE TO CLICK** Explorer -> data -> sample\_input.csv -> locate WTR-005 row -> pH field



Screenshot 114033 - WTR-005 missing pH



Screenshot 114044 - WTR-005 corrected

## DO THIS - CLICK BY CLICK

- 1 In Explorer, expand data/.
- 2 Click sample\_input.csv.
- 3 Locate the row whose sample\_id is WTR-005.
- 4 Find the pH position/column in that row.
- 5 Enter the approved valid pH value for the example.
- 6 Save the CSV with Ctrl+S / Cmd+S.
- 7 Re-check the row to ensure commas/columns were not accidentally shifted.

## EXPECTED RESULT

WTR-005 now contains a pH value and the CSV remains structurally valid.

## IF YOU DO NOT SEE THIS

If editing raw CSV text, be careful not to add/remove an extra comma. One misplaced delimiter can shift every later column.

# Rerun Until STEP 1 DATA CHECK PASSED

A clean pass closes the local validation portion of Step 1 - but Step 1 itself is not finished until the changes are committed and pushed.

**WHERE TO CLICK** VS Code terminal at repository root -> rerun validation command

Screenshot 114132 - passing validator

Screenshot 114157 - final pass state

## DO THIS - CLICK BY CLICK

- 1 Return to the terminal.
- 2 Run the same validator command again.
- 3 Confirm the shape/column diagnostics still look correct.
- 4 Confirm the missing-value assertion no longer fails.
- 5 Look for the final success text STEP 1 DATA CHECK PASSED.
- 6 Keep the terminal evidence visible while you move into Source Control; the changed CSV and validator must still be versioned and shared.

### TYPE / SELECT EXACTLY

**python src/validate\_data.py**

### EXPECTED RESULT

Terminal prints STEP 1 DATA CHECK PASSED.

### IF YOU DO NOT SEE THIS

If validation still fails, read the exact assertion/message and repair the data or specification issue it identifies. Do not proceed to Git staging until the local check passes.

### IMPORTANT

Local PASS is only a checkpoint. Phase 5 records and publishes the verified changes to the shared repository.

# Stage, Commit, Authenticate & Push

Convert the local validated state into shared Git history. This phase is still part of Step 1.

## By the end of this phase, students should be able to:

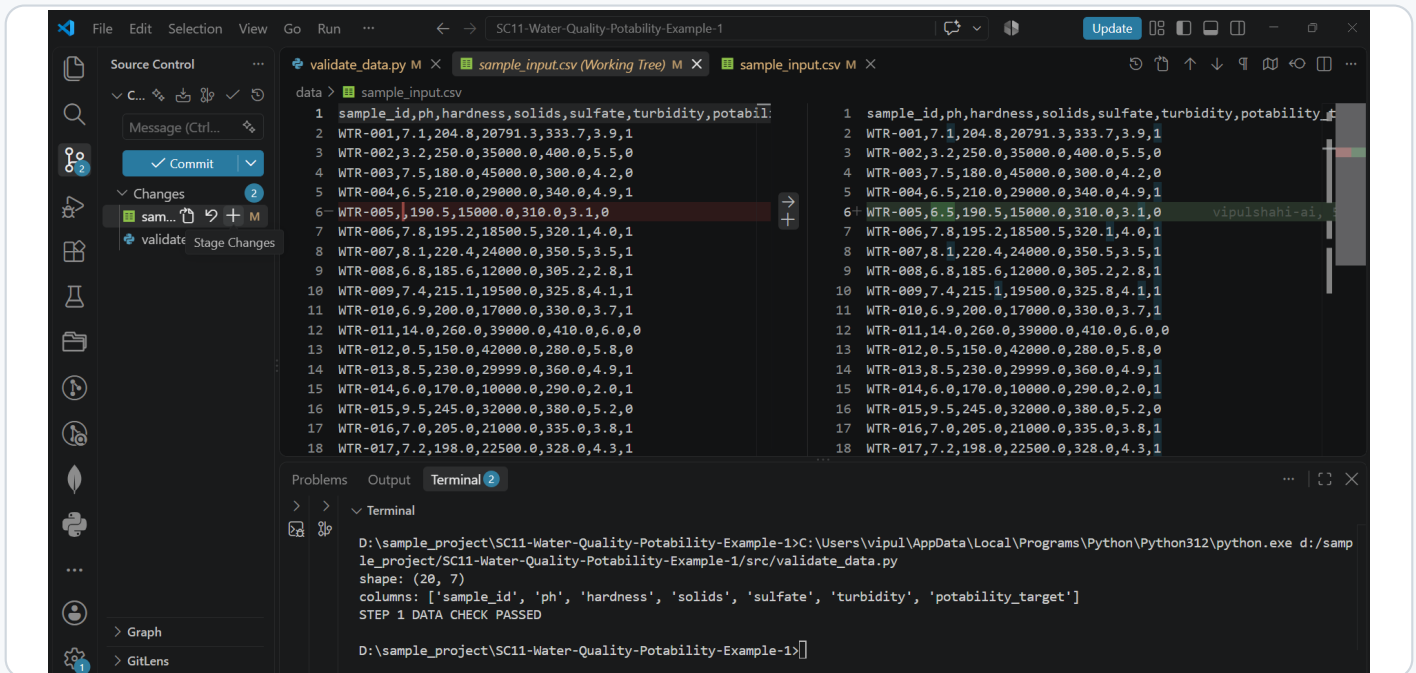
- 1 Review the exact line-level differences before committing.
- 2 Stage only the intended Step-1 files and write a meaningful commit message.
- 3 Push/Sync the local commit to origin/main.
- 4 Resolve wrong-account / missing-permission problems without automatically forking the team repository.

*Follow the screenshots and the click path exactly. Stop and verify each checkpoint before moving forward.*

# Open Source Control and Review the Diff

Never commit before checking what actually changed.

**WHERE TO CLICK** VS Code left Activity Bar -> Source Control icon -> Changes -> click sample\_input.csv



Screenshot 114230 - CSV diff in Source Control

## DO THIS - CLICK BY CLICK

- 1 Click the Source Control icon on the left Activity Bar.
- 2 Under Changes, locate data/sample\_input.csv.
- 3 Click sample\_input.csv to open the side-by-side diff.
- 4 Read the highlighted changed line and confirm it is the intended WTR-005 correction.
- 5 Also inspect src/validate\_data.py if it appears under Changes.
- 6 If unrelated files appear, open each one and decide whether it belongs to this Step-1 commit.

### EXPECTED RESULT

The diff shows only the intended data/code changes that produced the successful validation.

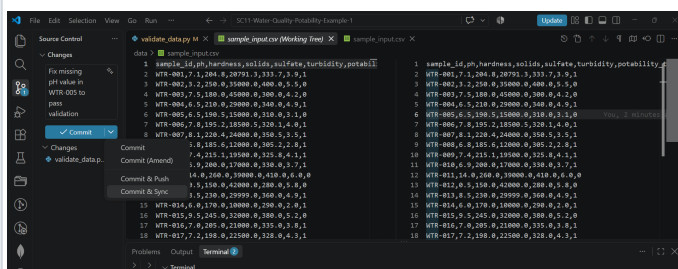
### IF YOU DO NOT SEE THIS

If you see temporary files, virtual environments, caches or unrelated experiments, do not stage them. Consider adding appropriate patterns to .gitignore if needed.

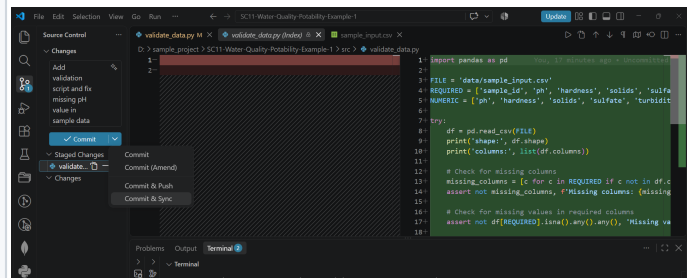
# Stage the Intended Files

Staging selects the exact snapshot that the next commit will contain.

**WHERE TO CLICK** Source Control -> Changes -> + icon beside each intended file (or right-click -> Stage Changes)



Screenshot 114513 - Source Control staging area



Screenshot 114646 - Staged Changes

## DO THIS - CLICK BY CLICK

- 1 Under Changes, move the mouse over data/sample\_input.csv.
- 2 Click the plus (+) icon to stage it.
- 3 Repeat for src/validate\_data.py if it was created/modified locally.
- 4 Expand Staged Changes and confirm the intended files moved there.
- 5 Leave unrelated files under Changes rather than staging them.
- 6 Re-open a staged diff if you want one final review before committing.

### EXPECTED RESULT

The files intended for the Step-1 update appear under Staged Changes.

### IF YOU DO NOT SEE THIS

If you accidentally stage the wrong file, use the minus/Unstage action to move it back to Changes before committing.

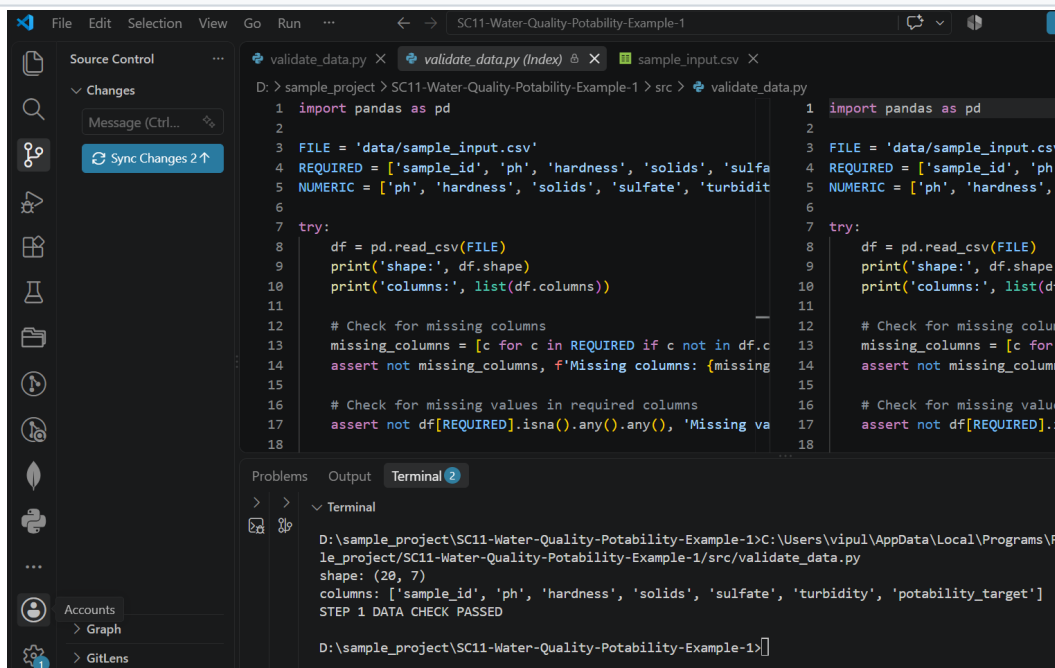
### IMPORTANT

Changed is not the same as Staged. Git commits the staged snapshot.

# Create a Meaningful Commit

The commit message should explain the outcome, not merely say “update”.

**WHERE TO CLICK** Source Control -> Message input at top -> type message -> Commit/checkmark button



Screenshot 114912 - Source Control commit/sync state

## DO THIS - CLICK BY CLICK

- 1 Click the Message input box at the top of Source Control.
- 2 Type a concise message that describes both the validator and starter-data correction.
- 3 Confirm Staged Changes contains exactly the intended files.
- 4 Click Commit (or the checkmark icon, depending on VS Code version).
- 5 Wait for the staged list to clear and for a Sync Changes / outgoing-commit indicator to appear.

### TYPE / SELECT EXACTLY

Add validation script and fix missing pH in

### EXPECTED RESULT

The changes are committed locally. Source Control may show an outgoing count or Sync Changes button, meaning the commit still needs to be pushed.

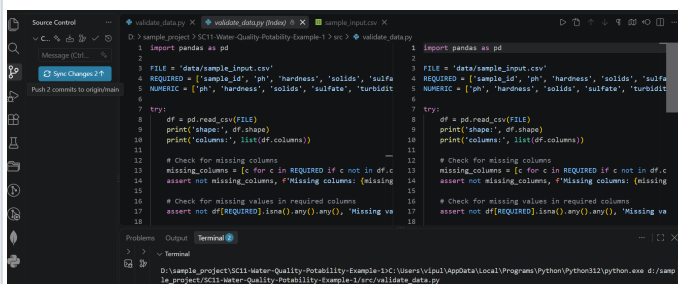
### IF YOU DO NOT SEE THIS

If Commit is disabled, check that at least one file is staged and that Git user.name/user.email configuration is available.

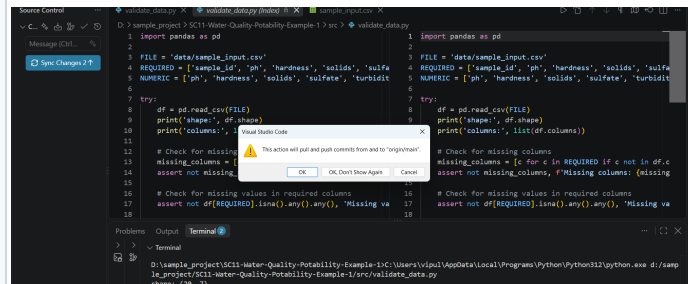
# Push / Sync the Local Commit

A local commit is not yet shared with the team.

**WHERE TO CLICK** Source Control -> Sync Changes / Push -> confirmation dialog -> OK



Screenshot 115039 - Sync Changes button



Screenshot 115050 - sync confirmation dialog

## DO THIS - CLICK BY CLICK

- 1 Look for the blue Sync Changes button or an outgoing-arrow count in Source Control.
- 2 Click Sync Changes (or Push if that is what your VS Code shows).
- 3 Read the confirmation dialog stating that VS Code may pull and push commits to/from origin/main.
- 4 Click OK to continue only after confirming origin/main is the intended team branch.
- 5 Wait for authentication or permission checks.

## EXPECTED RESULT

VS Code attempts to synchronize the local commit with origin/main.

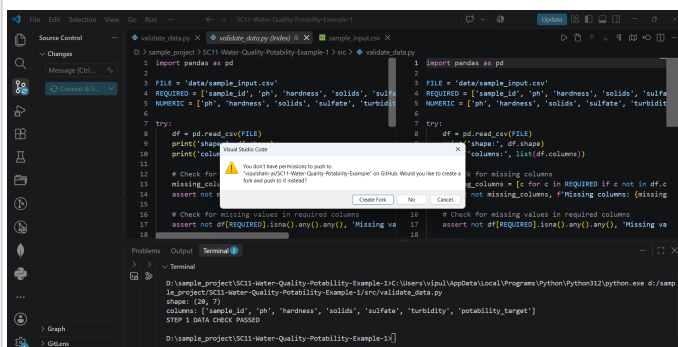
## IF YOU DO NOT SEE THIS

If a merge/pull conflict appears, stop and resolve it carefully with the team rather than blindly overwriting remote work.

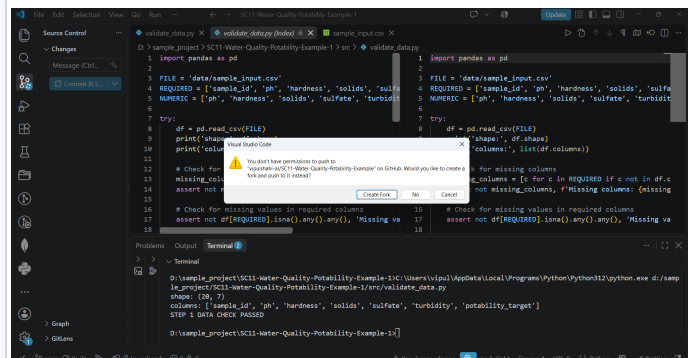
# Handle the No-Permission / Fork Prompt

For this team workflow, a fork prompt usually indicates the wrong account or missing collaborator access.

**WHERE TO CLICK** Permission dialog -> Cancel / No -> verify collaborator access and authenticated account



Screenshot 114716 - permission warning



Screenshot 114834 - fork prompt

## DO THIS - CLICK BY CLICK

- 1 Read the permission warning fully.
- 2 Do NOT automatically create a fork if this repository is supposed to be the shared team repository.
- 3 Choose Cancel/No to leave the push attempt without forking.
- 4 In GitHub browser, verify the same account is listed/accepted as a collaborator.
- 5 If the invitation is still pending, accept it before retrying.
- 6 Return to VS Code and verify which GitHub account is currently authenticated.

## EXPECTED RESULT

You avoid creating an unnecessary fork and move into correcting the actual access/account problem.

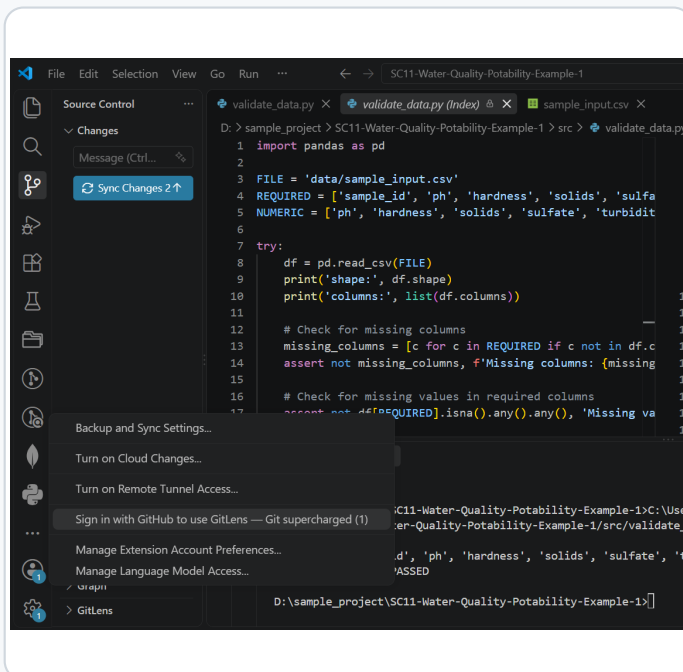
## IF YOU DO NOT SEE THIS

A public repository can be cloned by anyone, but push access still requires collaborator permission. Being able to view/clone the repo does not prove you can push.

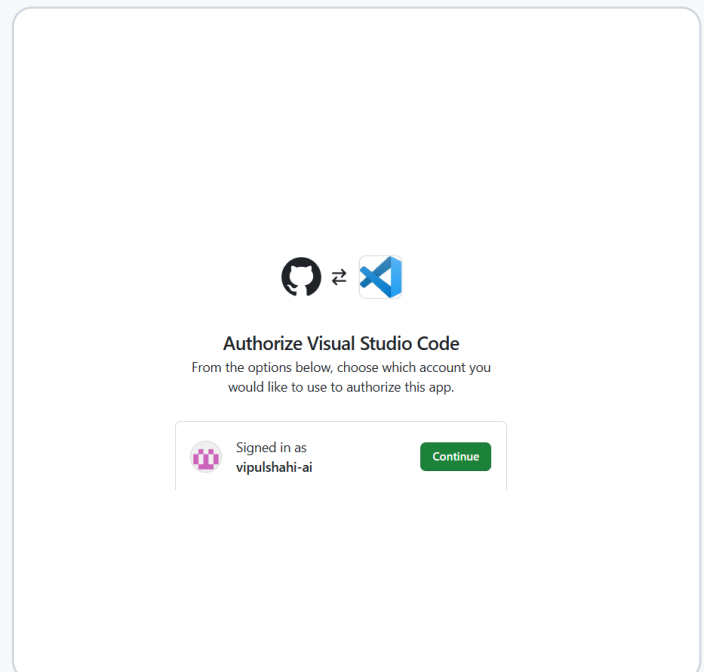
# Authenticate the Correct GitHub Account

Use the GitHub account that accepted the Phase-1 collaborator invitation.

**WHERE TO CLICK** VS Code Accounts / sign-in prompt -> GitHub sign-in -> browser authorization -> Continue



Screenshot 114949 - VS Code account controls



Screenshot 115007 - Authorize Visual Studio Code

## DO THIS - CLICK BY CLICK

- 1 Use the VS Code Accounts/profile control (typically bottom-left) or the sign-in prompt produced by Push/Sync.
- 2 Choose the GitHub sign-in/authentication option.
- 3 When the browser opens Authorize Visual Studio Code, check the displayed GitHub account.
- 4 If it is the account invited to the repository, click Continue/Authorize.
- 5 If it is the wrong account, switch/sign out before authorizing.
- 6 Return to VS Code after the browser reports authorization completed.

### EXPECTED RESULT

VS Code is authenticated to the same GitHub account that has collaborator write access.

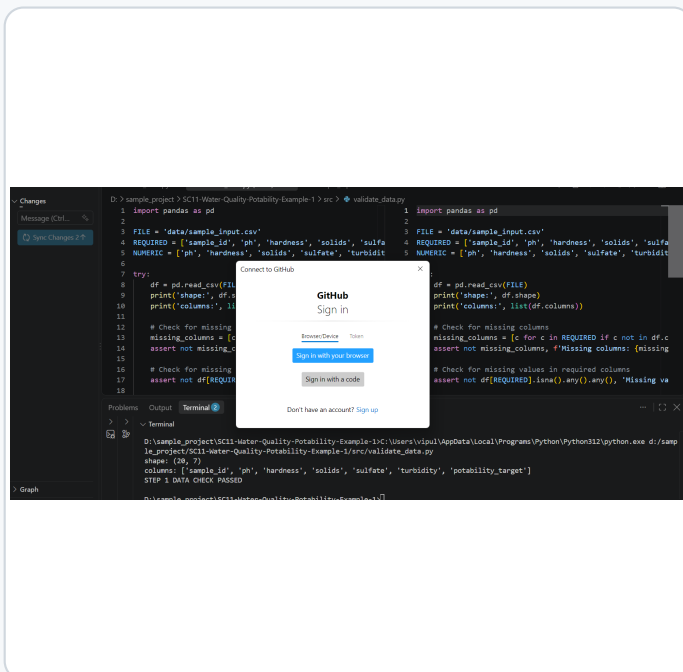
### IF YOU DO NOT SEE THIS

If the browser silently uses the wrong GitHub account, sign out of that browser session or open the authorization flow in a profile where the correct account is signed in.

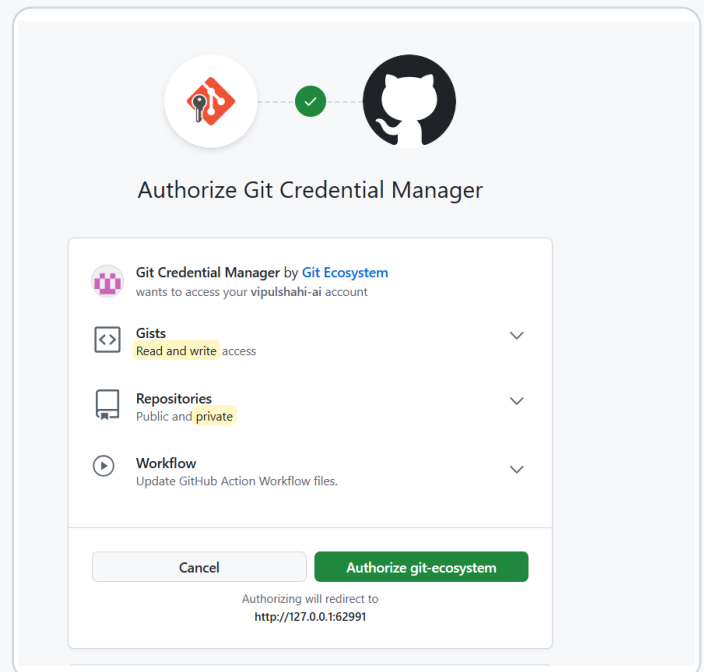
# Authorize Git Credential Manager and Retry

Credential Manager may require a second authorization step on Windows.

**WHERE TO CLICK** GitHub/Git Credential Manager prompt -> Authorize / Authorize git ecosystem -> return to VS Code -> Sync Changes



Screenshot 115355 - GitHub sign-in from VS Code



Screenshot 115407 - Git Credential Manager authorization

## DO THIS - CLICK BY CLICK

- 1 If Git Credential Manager opens, read the requesting application/account information.
- 2 Click the authorization option that allows Git operations for the correct GitHub account.
- 3 Complete any browser/device confirmation requested.
- 4 Return to VS Code.
- 5 Click Sync Changes / Push again.
- 6 Wait for the outgoing indicator to clear.
- 7 Optionally refresh the repository in a browser and verify the latest commit/files are visible remotely.

## EXPECTED RESULT

The validated CSV and validator commit are published to the shared GitHub repository; teammates can pull the same state.

## IF YOU DO NOT SEE THIS

If push still fails, re-check Phase-1 collaborator acceptance, repository URL/remote, account identity and branch. Do not create a new repository or fork as a shortcut.

# Product V1 Sketch & Step-1 Evidence

Store one tangible product-thinking artifact in the repository so Step 1 ends with visible evidence of what the future product should do.

## By the end of this phase, students should be able to:

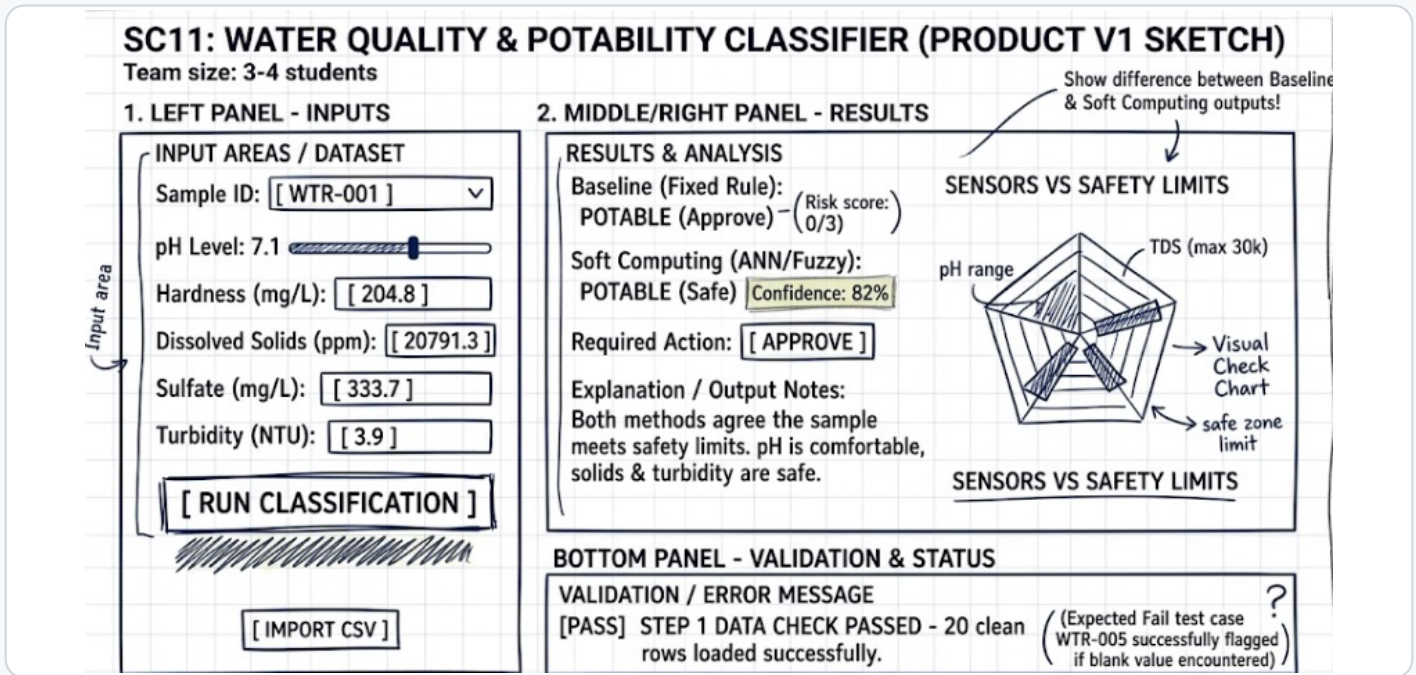
- 1 Prepare a Product V1 sketch showing inputs, outputs, comparison/analysis and validation/status areas.
- 2 Upload product-v1-sketch.png into docs/.
- 3 Commit the uploaded image to the repository.
- 4 Open the image in GitHub and confirm it renders before declaring Step 1 complete.

*Follow the screenshots and the click path exactly. Stop and verify each checkpoint before moving forward.*

# Prepare the Product V1 Sketch

The sketch is evidence of product thinking, not a polished final UI.

**WHERE TO CLICK** Create/export your sketch locally -> save as PNG with the approved filename



Screenshot 120543 - Product V1 Sketch

## DO THIS - CLICK BY CLICK

- 1 Make sure the sketch clearly shows what the user will input.
- 2 Show the baseline result area and the planned Soft Computing/model result area.
- 3 Include the main explanation/visualization region.
- 4 Include a validation/status/error-message area.
- 5 Add project-specific labels so a reviewer can understand the intended product without verbal explanation.
- 6 Export/save the image as product-v1-sketch.png.

**TYPE / SELECT EXACTLY**  
product-v1-sketch.png

### EXPECTED RESULT

A readable PNG sketch exists locally and communicates the first product concept.

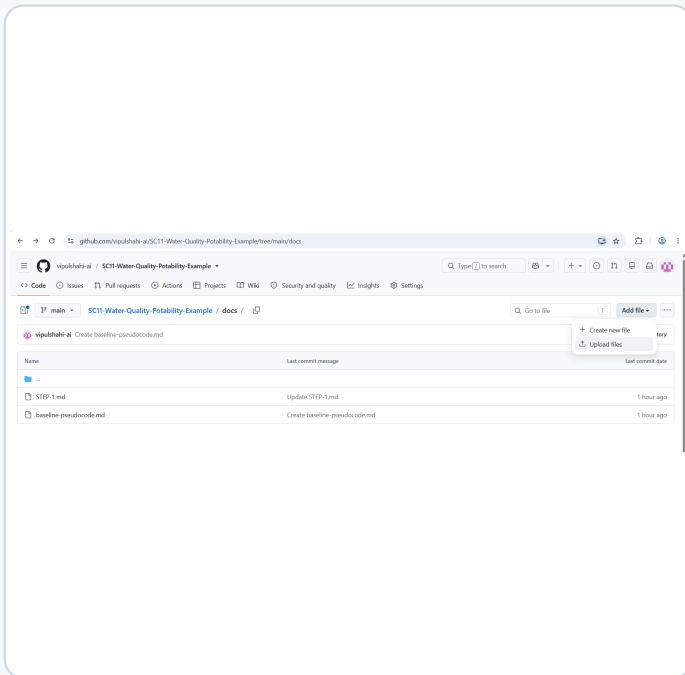
### IF YOU DO NOT SEE THIS

Do not submit an unlabeled wireframe with boxes only. The sketch should communicate user inputs, outputs and validation/status behavior.

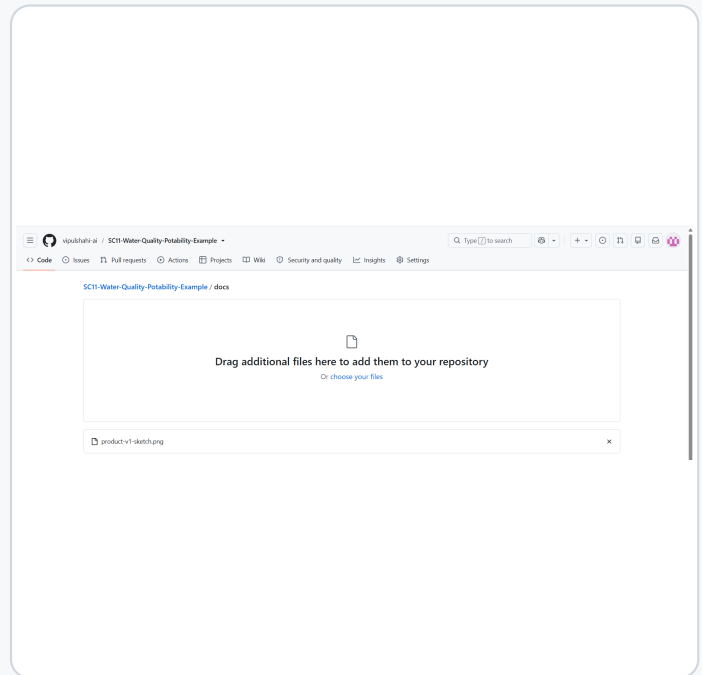
# Open docs and Start Upload

Upload the sketch into docs so it sits beside the Step-1 documentation.

**WHERE TO CLICK** Repository -> docs folder -> Add file -> Upload files -> choose your files / drag file



Screenshot 121026 - docs folder Add file



Screenshot 121111 - upload area

## DO THIS - CLICK BY CLICK

- 1 Open the repository Code page in your browser.
- 2 Click docs/.
- 3 Above the docs file list, click Add file.
- 4 Choose Upload files.
- 5 On the upload page, either drag product-v1-sketch.png into the large drop area or click choose your files.
- 6 If using the file picker, select product-v1-sketch.png and confirm/open it.
- 7 Wait until the filename appears on the upload page before committing.

### TYPE / SELECT EXACTLY

docs/product-v1-sketch.png

### EXPECTED RESULT

The upload page lists product-v1-sketch.png and it is being added inside docs/.

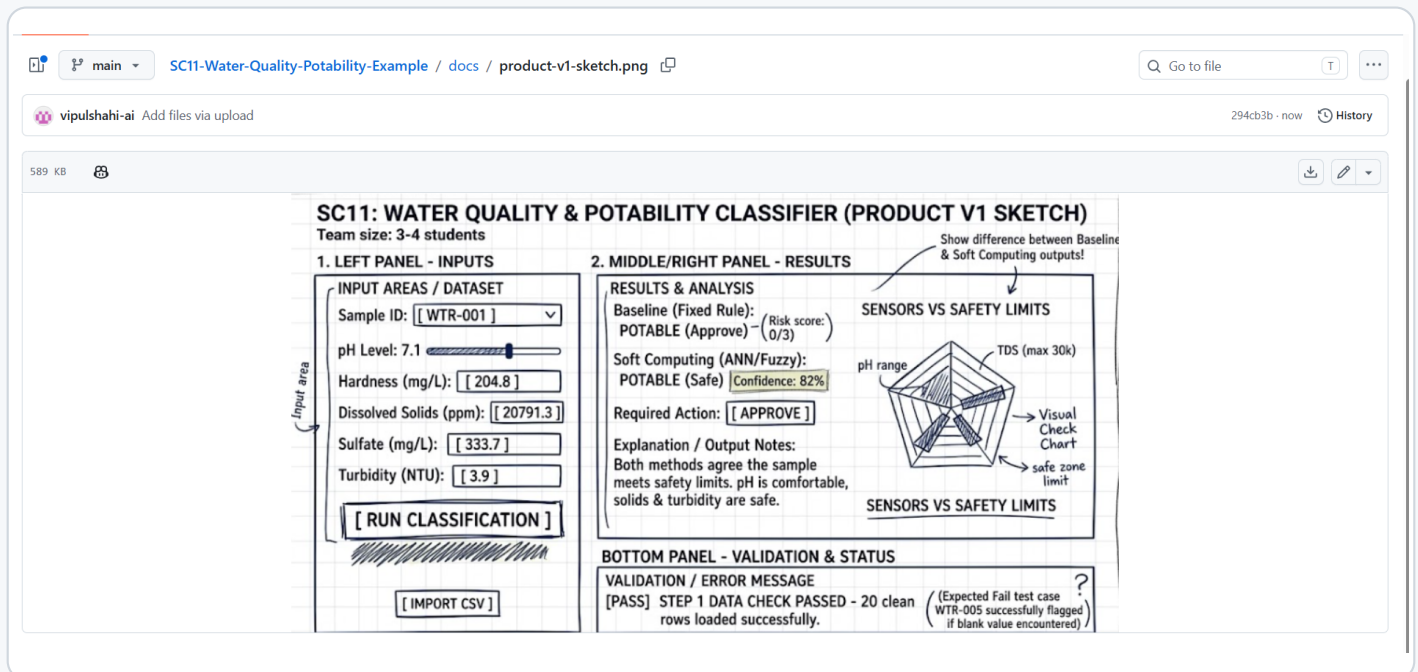
### IF YOU DO NOT SEE THIS

If you started upload from repository root, the image will land at root. Cancel and repeat from inside docs/ so the final path is docs/product-v1-sketch.png.

# Commit the Upload and Verify It Renders

The image is not part of Git history until the upload is committed.

**WHERE TO CLICK** Upload page -> scroll to commit section -> commit message -> Commit changes -> open image file



Screenshot 121202 - Product V1 sketch rendered in GitHub

## DO THIS - CLICK BY CLICK

- 1 After the file finishes uploading, scroll to the commit section at the bottom of the page if needed.
- 2 Enter/review a descriptive commit message such as Add Product V1 sketch.
- 3 Keep the Step-1 direct-to-main workflow selected.
- 4 Click Commit changes.
- 5 Return to docs/ and click product-v1-sketch.png.
- 6 Confirm GitHub renders the image clearly.
- 7 Use the breadcrumb to verify the final path is docs/product-v1-sketch.png.

### TYPE / SELECT EXACTLY

Add Product V1 sketch

### EXPECTED RESULT

docs/product-v1-sketch.png is committed on main and renders correctly in GitHub.

### IF YOU DO NOT SEE THIS

If the image is too small/blurred to review, replace it with a clearer export before closing Step 1.

## What “STEP 1 Complete” Means

Local validation alone is not the finish line. The shared repository must contain the complete evidence chain.

### Mandatory sequence

1. Repository + accepted collaborators -> 2. folder structure -> 3. Project Contract + Data Dictionary -> 4. starter CSV + placeholders -> 5. baseline pseudocode -> 6. local validation failure and correction -> 7. STEP 1 DATA CHECK PASSED -> 8. stage + commit -> 9. push/sync to shared GitHub -> 10. Product V1 sketch committed under docs/.

### Final completion gate

Do not mark Project Step 1 complete until the intended local code/data changes are visible in the shared repository and product-v1-sketch.png renders from docs/.

### Why this matters

A faculty reviewer should be able to inspect Step 1 from GitHub without depending on one student's laptop or verbal explanation.

## Expected Repository State

Use this as the final tree-level inspection before sign-off.

### Repository root

README.md (Data Dictionary)  
requirements.txt  
data/  
test/

### data/

README.md  
sample\_input.csv - 20 rows; corrected WTR-005 after the validation test

### docs/

STEP-1.md  
baseline-pseudocode.md  
product-v1-sketch.png

### src/

validate\_data.py

# Git Hygiene Before Final Sign-off

Commit the project evidence - not your machine clutter or secrets.

## Commit when changed

src/validate\_data.py; data/sample\_input.csv; intended Step-1 documentation; product-v1-sketch.png.

## Usually do NOT commit

.env/ or venv/; \_\_pycache\_\_/\*; \*.pyc; .env; passwords/tokens/API keys; unrelated screenshots; temporary exports; editor/system files; large raw datasets unless explicitly required.

## Commit-message standard

Use messages that describe the outcome, for example: "Add validation script and fix missing pH in starter data" or "Add Product V1 sketch". Avoid messages such as "update", "changes" or "final final".

# Final Reviewer Evidence Matrix

Every row below must have visible evidence in the shared GitHub repository.

## Problem + team responsibilities

docs/STEP-1.md is populated, readable and committed.

## Field definitions

Root README.md contains the rendered Data Dictionary.

## Starter data

data/sample\_input.csv contains the 20-row starter set and the WTR-005 correction is present after testing.

## Baseline plan

docs/baseline-pseudocode.md is present and readable.

## Data-quality proof

src/validate\_data.py exists; the local run produced STEP 1 DATA CHECK PASSED after detecting the planned failure.

## Version-control proof

The intended code/data changes are committed and pushed/synchronized to the shared repository.

## Product-thinking proof

docs/product-v1-sketch.png is committed and renders correctly in GitHub.

## Final sign-off condition

Project Step 1 is complete only when every evidence item above is visible and reviewable in the shared GitHub repository.